Governance Drift: Measuring Divergence Between Approved Intent and Operational Reality in Cloud-Native Systems

Obede Bessa
Independent Researcher — obedebessa@gmail.com

***Abstract*—Configuration drift compares desired and observed configuration; it cannot decide whether a running system remains covered by what authorized it. We define *governance drift* as a history-anchored relation among operational reality, current governance context, and a unique activated approval basis. The model separates five substantive components from epistemic observability and specifies stable identity, activation and supersession, and admissible temporal cuts. In a twelve-scenario closed world, a cumulative ablation raises unconditional exact-vector success from 75.0% for plane-local signals to 100% for the admitted-basis join. On a version-pinned Kind/Flux/Kyverno stack, 180 injections yielded 538/540 detections and 538/538 exact provisional class sets conditional on detection; 777 benign-transition polls produced no policy, authorization, intent, or environment drift, and all 60 trajectories ended provisionally consistent. A process-isolated audit reached the exact target in 27/27 correlated trajectories: cause-start-to-first-exact completion had median 4.465 s (P95 9.764), and cause-start-to-second-consecutive-exact completion had median 9.522 s (P95 19.811), without a Stable-VCL claim. A separate Argo CD/Gatekeeper replication produced 15/15 exact projected singleton classifications over declared components: S1 configuration and S3 policy used native paths, whereas S4 authorization reused the shared digest adapter. This is neither complete-vector validation nor cross-stack equivalence. The results demonstrate bounded semantic realizability and safe uncertainty handling, not natural prevalence or production effectiveness.**

***Index Terms*—Governance drift, configuration drift, GitOps, policy as code, cloud governance, Kubernetes, continuous compliance, DevOps, distributed systems.**

I. Introduction

Reconciliation-based delivery gave cloud operations a powerful invariant: the running system converges to a declared desired state, and divergence—*configuration drift*—is detected and repaired continuously [1, 2, 3, 4, 5]. The invariant is so useful that it has quietly become a proxy for a much stronger claim. When the reconciler reports SYNCED and the drift detector reports clean, operators, auditors, and dashboards read: *the system is in the state we approved*. This paper is about the gap between those two statements.

Consider a payments service whose deployment was approved on March 3rd: the manifest matched revision `rev-42`, satisfied policy version π_7, carried a service-owner approval bound to image digest `sha256:aaa...`, and traced to ticket `TKT-99`. Months later the cluster still matches its Git manifest byte for byte. Meanwhile: the security team superseded π_7 with π_8, which the running configuration violates; an emergency exception granted during an incident expired after four hours and has now been silently in force for six weeks; the registry re-tagged `svc:1.4.2` so the pods run a digest no approval ever referenced; and a well-meaning rollback reverted Git to `rev-37`, content whose approval belongs to a different change. Configuration drift: zero. One or more legitimacy conditions that covered the deployment on March 3rd no longer hold. We call this condition *governance drift*: the operational state has diverged not from its manifest but from the state, policy context, authorization basis, and intent *under which it was approved*.

The distinction is not terminological. The desired/observed configuration-drift signal implemented by reconcilers and commonplace IaC drift tools is a *two-place, memoryless* relation between two configurations and is therefore decidable by comparison [6, 7]. Configuration-management programs can retain approved historical baselines (Section IX), but their decision object remains configuration-scoped. Governance drift is a *three-place, history-anchored* relation among present state, evolving context, and an immutable admitted basis; we establish the separation in two complementary forms: constructed, tool-realizable counterexamples, and a definitional-dependency result showing that no detector whose inputs are configuration pairs—however sophisticated its diffing—can decide governance consistency, because the deciding facts are not functions of those inputs (Proposition 1). Worse, reconciliation can materialize an unauthorized desired-state change and then erase the transient configuration-drift signal, leaving intent drift: the rollback scenario above is converged into content outside the historically authorized change lineage (Section III).

Our executed study makes the separation concrete. We implement a governance-drift detector evaluated at cumulative evidence tiers—configuration comparison; plus policy re-evaluation; plus authorization records with validity semantics; plus artifact lineage; plus environment inventory against recorded assumptions—against twelve injected drift scenarios (including compound drift and evidence decay) and a no-drift control containing both runtime churn and *benign governance churn* (satisfied policy revisions, approved updates, hygienic exceptions), under a counterfactual scoring rule that credits in-

formation rather than alarm volume. The result is a strict detectability staircase (Table II): normalized configuration comparison detects exactly one scenario of twelve; each evidence tier detects precisely the scenarios whose deciding facts it carries; the governance tiers achieve zero false alarms *earned against* governance churn. A deliberately naive unnormalized comparator is retained only as a scoring diagnostic, not as a production baseline.

A. Research Questions

RQ0: Can divergence between approved intent and operational reality be formally represented and measured as governance drift?

RQ1: Which forms of governance drift can arise and be distinguished in cloud-native systems?

RQ2: Can governance drift be detected using existing operational, governance, and control-plane evidence streams?

RQ3: How quickly does governance drift become visible?

RQ4: Which forms of drift are detectable through configuration comparison alone, and which require authorization or evidence context?

RQ0 is answered by the formal model and the combined analytic and empirical results. RQ1 is answered by the taxonomy and scenario catalog (Sections III and VI); RQ2 and RQ4 by the dependency result (Proposition 1), the tiered architecture (Section V), the executed detectability study (Section VI), and the bounded live laboratory (Section VII); RQ3 by the study's event-time measurements and the laboratory's separated actuation, onset-to-detection, end-to-end, and time-to-evidence observations across three polling cadences.

B. Contributions and Epistemic Status

C1. Taxonomy (conceptual): formal definitions separating five substantive components—configuration, policy, authorization, intent, and environment—plus transversal epistemic evidence drift, with governance drift as their umbrella—and with the explicit commitment that governance drift must not, and provably does not, reduce to configuration drift (Section III).

C2. Formal model (conceptual/analytic): an immutable sequence of atomic approval snapshots plus separate activation and supersession records, stable unit identity, admissible cross-stream temporal cuts, observed state $G_{\mathrm{obs}}(t)$, time-indexed governance contexts $\mathscr{P}(t), \mathscr{A}(t), \mathscr{I}(t)$, and a component-wise governance-divergence vector $\mathsf{GD}_{\mathrm{sub}}(u,\theta)$ plus an observability mask $\mathsf{O}_{\mu}(u,\theta;T)$, with a reasoned rejection of a primitive scalar distance (Section IV).

C3. Separation results (analytic): constructed counterexamples and a definitional-dependency proposition establishing that configuration equality does not imply governance consistency and that no configuration-pair detector decides it—deliberately elementary, and billed as such (Sections III and IV).

C4. Detection architecture (design): a vendor-neutral tiered architecture mapping each component to a sufficient evidence tier, together with indistinguishable-world constructions proving tier minimality (Section V and Proposition 4).

C5. Executed detector study (empirical, closed-world): twelve scenarios $\times$ six detector tiers $\times$ twenty seeds, with counterfactual scoring, full-vector ground truth, three compound scenarios, exact-set and Hamming scoring, a separately implemented composite comparator without the admitted-basis join, and a control containing runtime *and* benign governance churn; all code and data packaged, matrix generated programmatically (Section VI). Its role is precisely scoped: it validates that the reference semantics realize the model's dependency structure and quantifies noise behavior; it is not field evidence, and it makes no claim about any real cluster.

C6. Repeated Kubernetes/GitOps laboratory (empirical, executed): a version-pinned Kind/Flux/Kyverno stack, local Git and OCI services, approved-state recorder, tiered evaluator, and executable injections for S1–S9. In one hundred eighty controlled injections (20 per scenario), three randomized-phase polling cadences produced 538/540 detections and 100% conditional exact-set accuracy for provisional class-set classifications under the laboratory's sequential-snapshot assumption. Twenty steady-state benign windows produced no alarms in 543 polls; a separate transition-inclusive campaign began three isolated observers before each benign mutation and retained all 777 polls, including seven expected configuration-convergence polls and 36 fail-safe epistemic warning polls but no policy, authorization, intent, or environment drift; all 60 trajectories ended with a provisionally consistent classification (Section VII). A secondary S10–S12 extension reached the exact provisional class set in 45/45 cadence observations; six separate 300-s benign windows added 4,680 steady-state polls without substantive or epistemic alarms. The execution demonstrates bounded realizability and behavior within the reported laboratory conditions, not natural occurrence, prevalence, production effectiveness, stream-loss reliability, or transfer.

C7. Boundary campaigns (empirical, executed): a nine-episode audit with 27 separate observer processes and 261 raw polls; nine controlled profiles through a two-hop localhost TCP evidence path; an in-memory batched-join benchmark through 1,000 units; and a live Kubernetes object-path campaign that completed 10- and 50-Deployment targets while enforcing declared stop rules. The process-isolated audit reached the exact target in 27/27 correlated trajectories. Cause-start-to-first-exact

completion had median 4.465 s (nearest-rank P95 9.764), and cause-start-to-two-poll exact persistence had median 9.522 s (P95 19.811). The latter is only a second consecutive exact poll, not watermark-qualified Stable-VCL. A bounded Argo CD/Gatekeeper replication (Section VII) produced 15/15 exact projected singleton classifications over declared evaluated components. S1 configuration and S3 policy used native second-stack paths; S4 reused the shared artifact adapter. Intent and environment were not evaluated, so the result is neither a complete formal vector evaluation nor a cross-stack equivalence test. The separate temporal-cut tests validate the admissibility contract rather than retroactively making the sequential live snapshots globally coherent. These campaigns test traceability, fail-safe transport semantics, decision-core scaling, live fan-out, and limited component-path transfer separately; none is presented as an equivalence, production-capacity, or reliability estimate.

Established results are cited; analytic claims are proved in the model; the closed-world and live-stack empirical claims are separated explicitly; and the repeated live-stack protocol is reported with its negative controls and misses rather than presented as production evidence.

C. Scope and Non-Goals

We address drift *after* an approved deployment; admission-time enforcement [8, 9, 10, 11] is complementary and assumed imperfect (break-glass paths exist by design). We propose no product; the detector logic we release is a reference semantics, not a system. And we take no position on *which* governance controls organizations should impose—the model makes drift relative to whatever was approved under whatever policy; the choice of policy is the organization's.

II. Background: The Reconciliation Blind Spot

A. Desired-State Delivery and Its Invariant

GitOps-style delivery declares desired state in a versioned repository and runs controllers that continuously converge the runtime toward it [1, 2, 3, 4]; operational practice treats the resulting convergence signal as a primary health indicator [12]. The reconciler's contract is a fixed-point property over *configurations*: $G_{\text{obs}} \rightarrow$ desired. Drift detection in common desired-state practice—reconciler status, cloud configuration recorders, IaC drift tools [6, 7], and posture management [13]—implements variations of this comparison. Configuration-management standards also preserve approved baselines and controlled change history [14, 15]; that stronger tradition is configuration-scoped and is treated explicitly in Section IX.

Three properties of the desired/observed regime matter here. First, the comparison's *reference* is the current desired state, not the approved state: if the desired state itself changes illegitimately, the reconciler converges to the illegitimate state and reports health. Second, the comparison's *vocabulary* is configuration: facts that are not configuration—policy versions, approval windows, artifact lineage—are outside its type. Third, this pair comparison is *memoryless*: it relates two present states and consults no past event, whereas approval is precisely a past event.

B. The Governance Context Moves

Meanwhile, the context that made a deployment legitimate evolves on its own clocks. Policy-as-code is versioned and revised continuously [9, 10]; organizations supersede controls without redeploying workloads, under regulatory regimes that presume change remains authorized over time [16, 15]. Authorizations are temporal by nature—emergency exceptions with windows, approvals with scopes, roles that are granted and revoked [17, 18]; their validity changes with no configuration event at all. Artifact references are mutable (tags move; registries re-tag), so the binding between an approval's subject and the running binary can rot while every manifest byte stays fixed [19, 20]. And parts of the environment that govern a workload's risk (IAM scope, network exposure, cloud-resource settings) are frequently *unmanaged*: no desired state exists for them, so no comparison can flag their change.

C. Why This Produces a Blind Spot, Not a Bug

None of the mechanisms above is broken. Each does exactly what it is specified to do; the blind spot is an emergent property of their composition. The property stakeholders actually care about—*this running state is one that current policy, valid authorization, and recorded intent still cover*—is commonly checked jointly at admission and thereafter only in fragments: policy engines background-scan existing resources against *current* policy [10, 9], configuration recorders track unmanaged resources against their own baselines [6], and continuous-validation features re-check artifact policy [11]—each against its own reference. In our bounded source set, these mechanisms evaluate fragments of the relation; none of their documented native decision objects jointly evaluates runtime state, current context, and a uniquely activated admitted basis while returning class-resolved verdicts. Prior uses of "governance drift" cover vendor-managed organizational controls, cross-cloud policy consistency, and behavioral governance of AI systems (Section IX); we neither claim to coin the phrase nor treat those uses as identical. The narrower systems property studied here—joint consistency of running state with current context *and* an activated, recorded approval basis, with class-resolved verdicts—is not supplied by any one of those formulations or by the mechanisms above. This paper's contribution is its deployment-level three-place model, evidence contracts, and detector semantics; each existing fragment supplies machinery that one tier can reuse (Section V).

A terminological note to prevent collisions: "drift" is also used for statistical distribution change in learning systems (concept drift [21]); our subject is unrelated. Industry writing on "cloud governance and drift" uses governance as the *program* that manages configuration drift [22]; our subject is drift *of governance itself*—a relation the configuration vocabulary cannot express, as Section III now makes precise.

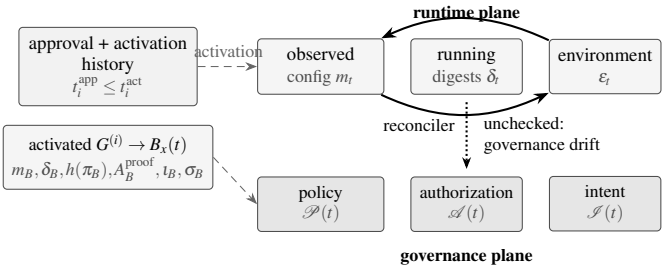

Fig. 1. Approved intent versus runtime reality. The approval event at t_0 freezes a snapshot: configuration, policy version, authorization basis, intent linkage, and environment assumptions. Afterward, two planes evolve independently: the runtime plane (observed configuration, running artifacts, environment) and the governance plane (policy versions, authorization validity, intent records). Reconcilers continuously compare within the runtime plane (solid loop). Mechanisms in our bounded source set evaluate portions of the dashed relation, but none of their documented native decision objects joins both planes to a uniquely activated admitted basis and returns the full class-resolved verdict—the gap studied here.

III. A Taxonomy of Drift (RQ1)

We fix vocabulary relative to an *approval event*: the moment a change was legitimized for an environment. Figure 1 shows the two planes that subsequently evolve. All definitions are made precise in the model of Section IV; here we give their content and their separations.

Definition 1 (Approved State)*: The* approved state *of a deployment is an append-only sequence of immutable snapshots, one per approval event. Each snapshot records: the approved configuration (manifest content and the artifact digests it resolves to), the policy version under which it was evaluated, immutable authorization proof (approvals, exceptions, their subjects, scopes, execution times, and windows), the intent linkage (the change record and revision the approval covers), and the declared environment assumptions within which the risk assessment was made. Approval appends a record; activation binds it to an effect that became operational. Evaluation selects the unique maximal activated covering successor, never merely the latest approval timestamp; pending, aborted, superseded, and prior snapshots remain evidence rather than being overwritten (Proposition 2).*

Definition 2 (Configuration Drift)*: Divergence between the* current desired *configuration and the* observed *configuration:* $G_{\text{obs}}(t) \neq \textit{desired}(t)$*. This is the classical notion [6, 7, 14], decidable by comparison of present states. It is an operational component of the umbrella vector; by itself it does not assert that either state is unauthorized.*

Definition 3 (Policy Drift)*: Divergence between the policy version* π_B *pinned in the selected admitted basis* $B_x(t)$ *and the policy context now governing the environment,* as it bears on this deployment*:* $\mathscr{P}(t)$ *has changed such that the running configuration would not be admitted under it today, although it was compliant when approved.*

Definition 4 (Authorization Drift)*: Invalidation or mismatch of the authorization basis without any configuration event: a continuing approval or temporary exception lapsed while its effect persists; an applicable approval was revoked under its declared prospective or retroactive semantics; or the running artifact's digest is not the subject any valid approval refers to. A one-shot approval need only have been valid at execution, provided its integrity-valid proof survives.*

Definition 5 (Intent Drift)*: Divergence between the lineage now driving the runtime and the historically authorized change lineage recorded by the selected admitted basis: the desired state was changed (e.g., rolled back) to content that was never covered by that change or belongs to a different change, and reconciliation faithfully executes it. Expiry or prospective revocation changes current authorization, not the historical fact of what intent was authorized. Only an explicitly retroactive revocation whose semantics invalidate the original approval can invalidate that historical intent relation.*

Definition 6 (Evidence Failure and Evidence Drift)*: Failure of any declared component evidence contract—including the admitted basis or its activation order, authorization proof or required live status, policy result, intent/lineage chain, environment inventory, freshness, authenticity, subject linkage, or stream health—so the corresponding governance relation can no longer be established. For component* j*, let*

$$\text{EvdFail}_j(t) \iff o_j(t) = 0,$$
$$\text{EvdDrift}_j(t) \iff o_j(t^-) = 1 \wedge o_j(t) = 0.$$

The first is current unavailability; the second is the witnessed transition from answerable to unanswerable. Without a prior observation, the system may claim evidence failure or low coverage, not a temporal drift it did not witness. Both make the affected relation undecidable *rather than false and share the operational label* evidence*, because both require evidence repair and withholding a polar verdict. This label is therefore an* epistemic, transversal *class: unlike the five substantive components, it does not assert that a governed relation is inconsistent; it records that one or more such relations can no longer be decided. We keep it inside the governance-drift umbrella because the operational response—repair evidence and withhold a polar verdict—is class-specific.*

Definition 7 (Environment Drift)*: Violation, outside the managed configuration's scope, of an environment predicate recorded by the approval's risk assessment: IAM scope broadened, network exposure altered, an adjacent cloud resource modified out of band—no desired state exists for the changed property, so no configuration comparison can even represent the divergence.*

Definition 8 (Governance Drift)*: The condition in which the operational state of an environment diverges from the state, policy context, authorization basis, intent, or environment assumptions under which it was approved, together with the epistemic condition in which one or more such relations can no longer be decided. Formally, the umbrella contains five sub-*

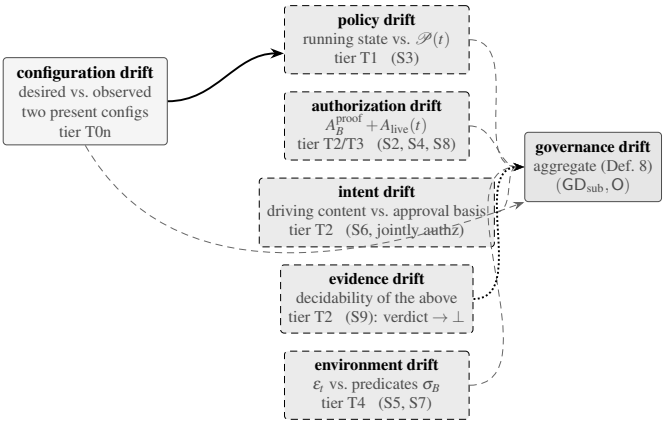

Fig. 2. The drift taxonomy. Configuration drift (left) is a relation between two present configurations. Five substantive components relate the present state to the approved snapshot or evolved context; evidence drift is the transversal loss of decidability shown by the dotted path. Their umbrella is governance drift (Definition 8). Annotated under each class: the evidence tier of Section V that first decides it, as measured in Section VI.

stantive divergence components (Definitions 2 to 5 and 7) and one transversal evidence condition (Definition 6). A deployment is governance-consistent *at t when every substantive divergence of Section IV is zero and every required component is decidable. The substantive components are not mutually exclusive: one incident may drift several at once (an unapproved rollback drifts intent* and *leaves the runtime on digests no valid approval covers), which is one reason the model of Section IV keeps them as a vector.*

A. Risk Channels and Severity

The five substantive divergence components and the epistemic evidence class are *not* equivalent in ontology or consequence. A raw count therefore cannot stand in for severity. Each class opens a different primary risk channel and implies a different default disposition, as summarized in Table I. Configuration drift first threatens operational integrity; policy drift threatens compliance with the currently governing control; authorization drift threatens security legitimacy; intent drift threatens the legitimacy of the change itself; evidence drift threatens audit and forensic decidability; and environment drift changes the exposure or assumption boundary around the approved system.

This mapping is diagnostic and decision-oriented, not ordinal: each class is associated with a primary risk channel and candidate response family; causal impact and final disposition require incident-specific evidence. Within a class, severity must still be conditioned on asset criticality, affected subjects, exposure time, reversibility, and the confidence permitted by surviving evidence. Across classes, a low-impact authorization mismatch can be less severe than a configuration fault that stops a safety-critical service. The taxonomy thus supports class-specific prioritization without pretending that one universal weight orders every incident.

B. The Central Separation

The taxonomy's load-bearing claim is that Definition 8 does not reduce to Definition 2. We establish it constructively and then in general form.

1) Constructed counterexamples

Figure 3 presents the canonical scenario in full: a deployment whose observed configuration equals its desired configuration at every instant shown, and which passes from governance-consistent to governance-inconsistent three separate times—at the policy supersession (policy drift), at the exception expiry (authorization drift), and at the registry re-tag (authorization drift at digest granularity). In the fourth panel the desired state itself is rolled back and the reconciler *eliminates* the transient configuration drift by converging to content with no valid approval basis: reconciliation can materialize a desired-state change outside the historically authorized lineage and then erase the transient configuration-drift signal, leaving intent drift. Each panel is realizable with standard tooling, and each is one of the executed scenarios of Section VI.

2) The dependency form

Proposition 1 (Configuration equality does not decide governance consistency)*: Let D be any detector whose inputs are configuration pairs $(\mathit{desired}(t), G_{\text{obs}}(t))$—including their full histories. There exist worlds w_1, w_2 with identical configuration-pair histories in which the same deployment is governance-consistent in w_1 and governance-inconsistent in w_2. Hence no such D decides governance consistency, for any diffing sophistication.*

Proof. Take w_1 and w_2 to agree on all configurations at all times and differ only in the governance plane: in w_2 the policy version governing the environment was superseded after t_0 by one the running configuration violates (or: an exception's window elapsed; or: a valid approval in w_1 is absent in w_2 for the same content). These differences live in $\mathscr{P}(t), \mathscr{A}(t), \mathscr{I}(t)$, which are not functions of the configuration pair; both worlds present D with identical inputs, and Definition 8 assigns them different values. D outputs the same answer in both, so it errs in one. □

The proposition is elementary—a definitional-dependency observation, and we bill it as such rather than as a deep impossibility theorem. Its force is architectural: it says deployed drift detectors are not approximations of governance-drift detectors but detectors *of a different relation*, and no incremental improvement of configuration diffing closes the gap *as a decision procedure*. Two bounds on what it claims: it rules out *deciding*, not statistical *hinting*—in real estates, governance-plane events often correlate with configuration-plane events (policy revisions ship alongside config pushes; rollbacks are visible in Git history), so configuration histories may carry probabilistic signal about governance state; how much is an empirical question for the laboratory, which our closed world deliberately does not answer. And it says nothing against comparison as a *mechanism*: the governance tiers of Section V are themselves

TABLE I
PROPOSED CLASS-RESOLVED RISK CHANNELS AND ILLUSTRATIVE DEFAULT RESPONSE FAMILIES. THESE ROWS ARE NOT EMPIRICALLY VALIDATED OR A SCALAR RANKING: ACTUAL SEVERITY DEPENDS ON BLAST RADIUS, EXPOSURE DURATION, REVERSIBILITY, CONSEQUENCE, AND EVIDENCE COVERAGE.

Drift class	Primary risk channel	Governance question	Default response family
Configuration	Operational availability and integrity	Does observed state still match the managed desired state?	Reconcile, contain, or accept an explicit runtime exception
Policy	Compliance and control validity	Would the running state satisfy the policy governing it now?	Re-evaluate, migrate under an SLA, or document a current exception
Authorization	Security and privilege legitimacy	Is the running subject still covered by valid authority?	Revoke, block, contain, or page the accountable owner
Intent	Governance and change legitimacy	Does the deployed revision implement the change that was approved?	Halt propagation, restore the authorized revision, or obtain new approval
Evidence	Audit and forensic decidability	Can the approval basis and lineage still be established?	Preserve records, escalate, and report *undecidable* rather than compliant
Environment	Assumption and exposure boundary	Do unmanaged surroundings still satisfy the recorded risk assumptions?	Contain exposure and reassess the approval in the changed environment

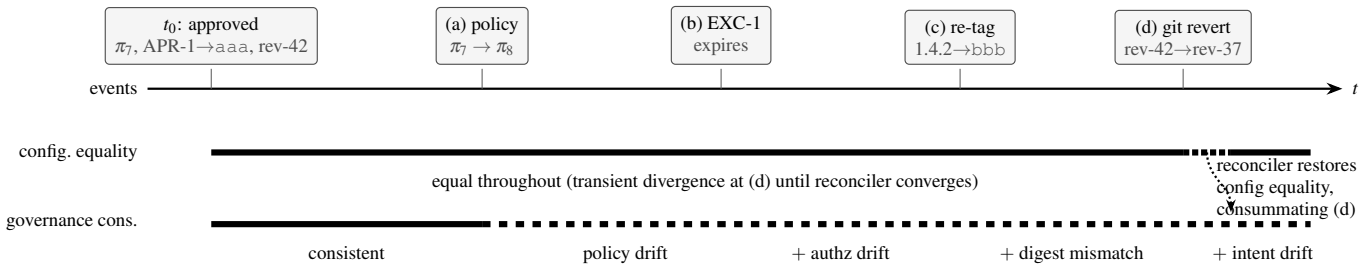

Fig. 3. Configuration-consistent but governance-inconsistent, in four panels. Timeline of one deployment: observed configuration (bottom band) remains equal to desired configuration throughout—configuration drift is zero at every instant after reconciliation—while governance consistency (top band) is broken successively by (a) policy supersession $\pi_7 \to \pi_8$, (b) expiry of emergency exception EXC-1 whose effect persists, (c) registry re-tag moving `svc:1.4.2` to a digest no approval references, and (d) a Git rollback that the reconciler faithfully converges to, eliminating transient configuration drift while stranding the runtime on content whose approval belongs to a different change. Panels correspond to Scenarios S3, S2, S4, and S6 of Section VI.

comparisons—against recorded bases, validity clocks, and coverage sets—which is precisely the point: the input vocabulary, not the mechanism, is what must widen.

3) What configuration drift remains

The separation does not demote configuration drift: unauthorized manual change (Section VI, S1) is both configuration and governance drift, and configuration comparison is its right detector. The taxonomy's point is containment, not replacement: configuration drift is one of five substantive components under the six-class umbrella, and the only one decidable in the configuration vocabulary—which the study confirms empirically as a detectability staircase whose first step is exactly that one class.

IV. A FORMAL MODEL OF GOVERNANCE DRIFT (RQ0, RQ2, RQ4)

A. States and Contexts

Fix a deployment x in environment e. Snapshot i is approved at t_i^{app} and, if its authorized effect becomes operational for (x,e), activated at $t_i^{\text{act}} \geq t_i^{\text{app}}$.

Stable unit reference. Every joined record is keyed by $u = \text{UnitRef}(x,e)$, an opaque, collision-resistant identifier minted once for the governed workload lineage in that environment. A mutable name, label, or current orchestrator UID is not a unit reference. Object recreation continues the same u only through an integrity-valid continuation edge from the prior runtime identity; name reuse or a copy without that edge creates a new u. Approval, activation, observation, policy-result, authorization, intent, lineage, and inventory records all carry u. If a runtime subject maps to zero or multiple unit references, every component requiring that join is undecidable rather than attached by name. We retain x in the notation below, but all joins use this stable reference.

Approved-snapshot history. Per Definition 1, every approval event appends an immutable snapshot

$$G_{\text{app}}^{x}(t) = \{G_{\text{app}}^{(0)}(t_0^{\text{app}}), \ldots, G_{\text{app}}^{(k)}(t_k^{\text{app}})\},$$
$$G_{\text{app}}^{(i)} = \big(m_i, \delta_i, h(\pi_i), A_i^{\text{proof}}, \iota_i, \sigma_i\big).$$

The append is atomic for one u: a partial tuple is not a candidate basis. Each record contains the approved manifest content m_i and artifact digests δ_i it resolved to at approval; the pinned policy version; the authorization basis A_i^{proof} (the immutable proof that approval instruments covered the named subjects and scope at their execution times); the intent linkage ι_i (change record, revision); and the environment assumptions σ_i (the declared properties of e—identity scopes, exposure—under which risk was assessed). Each record also carries scope, lifecycle state, and an explicit Supersedes edge for overlapping approvals. Approval alone does not change the running deploy-

ment's basis. Activation is the durable event that binds the authorized transition to the effect that actually became operational; a pending or cancelled rollout therefore cannot displace the previous basis. Aborted_i is terminal only before activation; a rollback after activation is a new authorized transition and snapshot, not a silent reactivation of an older record.

Let $t_i^{\text{act}} = \infty$ until activation and define

$$\mathscr{C}_{x,e}(t) = \{i : t_i^{\text{act}} \le t \ \wedge\ \text{Scope}_i(x,e) \ \wedge\ \neg\text{Aborted}_i\}.$$

Write $i \prec_{x,e} j$ when j transitively supersedes i over (x,e). If $\mathscr{C}_{x,e}(t)$ has one $\prec_{x,e}$-maximal element, call it $b(t)$; if it has none or multiple incomparable maxima, set $b(t) = \perp$ and make every basis-dependent component undecidable. The admitted basis is

$$B_x(t) = \begin{cases} G_{\text{app}}^{(b(t))} = \big(m_B, \delta_B, h(\pi_B), A_B^{\text{proof}}, \iota_B, \sigma_B\big), & b(t) \neq \perp; \\ \perp, & b(t) = \perp. \end{cases}$$

Thus "latest approved" is not the rule: the selected record is the unique *maximal activated successor* for this scope. Selection does not require current authorization validity: an expired or revoked basis must remain selected so its divergence can be reported. Superseded snapshots remain available for audit and lineage; benign reapproval appends a new basis rather than mutating history. Append-only means records cannot be rewritten, not that their storage cannot be lost; loss of A_B^{proof} would be an evidence failure. S9 instead preserves that proof and removes authenticated live status required by continuing authority.

Proposition 2 (Activation-safe basis selection)*: If the activated candidates covering (x,e) are nonempty and form a finite chain under the acyclic transitive closure of* Supersedes*, $B_x(t)$ is unique. Appending a pending or aborted approval cannot change it. An ambiguous overlap is observable as $b(t) = \perp$, never resolved by timestamp.*

Proof. A finite nonempty chain has one maximal element. Pending and aborted records are outside $\mathscr{C}_{x,e}(t)$; therefore appending either leaves its maximal element unchanged. Multiple maxima supply no model-grounded tie-breaker and consequently fail the basis evidence contract. □

Observed state. $G_{\text{obs}}(t) = (m_t, \delta_t, \varepsilon_t)$: the observed configuration, the digests actually executing, and the observed environment properties needed to evaluate the predicates in σ_B.

Contexts. Three time-indexed context streams: $\mathscr{P}(t)$, the policy version(s) governing e at t with their evaluation semantics; $\mathscr{A}_{\text{live}}(t)$, the authenticated current administrative state of instruments (revocations, continuing validity, expiry); and $\mathscr{I}(t)$, the intent records and the desired-state revision currently driving delivery. Evidence availability, where needed, is a fourth stream $\mathscr{E}(t)$ (the records from which the above can be established).

Admissible temporal cuts. Stream access alone does not license a join across unrelated capture times. A record r from source s carries a capture interval $I(r) = [c_r^-, c_r^+]$ in a common reference-time domain. The interval includes capture duration and expands the source timestamp by its declared clock-error bound κ_s. At evaluator observation time T, a source watermark $w_s(T)$ is a conservative capture-lower-bound frontier: it promises that no later-arriving record can have $c_r^- \le w_s(T)$. For component j, let S_j be its required sources, F_s their published source-freshness bounds, Δ_j its maximum permitted cross-source temporal spread, and F_j^{cur} the maximum age of a cut that may be labeled current. For logical cut θ, let $\mathscr{R}_s(u)$ be the records from s linked to u, and let $r_s(u,\theta)$ be the unique latest record for u from s with $c_{r_s}^+ \le \theta$. The cut is admissible exactly when

$$\begin{aligned} \text{AdmCut}_j(u,\theta;T) \iff & \bigwedge_{s\in S_j} \big[\text{UnitRef}(r_s) = u \wedge c_{r_s}^+ \le \theta\big] \\ & \wedge \bigwedge_{s\in S_j} \big[\theta \le w_s(T) \wedge \theta - c_{r_s}^+ \le F_s\big] \\ & \wedge \bigwedge_{s\in S_j} \big[\nexists r \in \mathscr{R}_s(u) : c_r^- \le \theta < c_r^+\big] \\ & \wedge \left[\max_{s\in S_j} c_{r_s}^+ - \min_{s\in S_j} c_{r_s}^- \le \Delta_j\right]. \end{aligned}$$

This predicate establishes a coherent *historical* as-of-θ join; it does not by itself make that join current at observation time T. A verdict labeled current must additionally satisfy

$$\begin{aligned} \text{CurCut}_j(u,\theta;T) \iff & \text{AdmCut}_j(u,\theta;T) \\ & \wedge [0 \le T - \theta \le F_j^{\text{cur}}] \\ & \wedge \bigwedge_{s\in S_j} [0 \le T - c_{r_s}^+ \le F_s]. \end{aligned}$$

For later definitions, write

$$\text{ModeCut}_{\mu,j}(u,\theta;T) = \begin{cases} \text{AdmCut}_j(u,\theta;T), & \mu = \text{historical}; \\ \text{CurCut}_j(u,\theta;T), & \mu = \text{current}. \end{cases}$$

Thus watermarks close the prefix, capture intervals make clock skew explicit, and Δ_j prevents a verdict from mixing evidence too far apart in time. A source that exposes only a capture-upper-bound watermark cannot establish this closure: a later interval could still straddle θ, so the affected cut is undecidable. A missing clock bound, record, unique latest record, or watermark—or an interval that straddles the requested cut—makes the affected component undecidable. An evaluation that satisfies only AdmCut is explicitly reported as historical at θ and makes no claim about state at T; a current evaluation requires CurCut for every reported component. The evaluator never manufactures a polar verdict from an inadmissible or stale temporal mixture. Every occurrence of t in a component relation below denotes a common logical cut admitted by that component's declared evaluation mode.

Atomic bundles and composition. A published verdict bundle is atomic only for one unit and one cut and declares $\mu \in \{\text{historical}, \text{current}\}$:

$$\begin{aligned} \mathscr{V}_{u,\theta;T}^{\mu} = \big(& u, \theta, T, \mu, \text{id}(B_x(\theta)), \\ & \mathsf{GD}_{\text{sub}}(u,\theta), \mathsf{O}_{\mu}(u,\theta;T), \\ & \{\text{id}(r_s), I(r_s), w_s(T)\}_s\big). \end{aligned}$$

Its fields are integrity-bound and a partial publication is invalid; the underlying sources need not commit in one transaction because AdmCut supplies temporal coherence. One approval transaction may emit several per-unit snapshots carrying a common parent bundle identifier, but that does not silently make a multi-workload release atomic. A compound claim composes per-unit bundles only when each bundle is valid and their cut constraints admit a common θ; otherwise the compound claim is undecidable while independently valid per-unit verdicts remain reportable. Declared cross-unit dependency edges determine where such failure propagates.

Authorization semantics. Every instrument a declares $\mathrm{mode}(a) \in \{\text{one-shot}, \text{continuing}, \text{temporary-exception}\}$, subject, scope, validity interval, and revocation policy. One-shot authority is required at execution and its proof is retained afterward; continuing authority must remain valid while its effect persists; a temporary exception is continuing only until its expiry. Revocation is prospective unless the record explicitly declares retroactive invalidation. Write $\mathrm{Retro}(a)$ when that exceptional policy is declared, $\mathrm{ProofObs}(a,t)$ when the immutable execution proof is available, and $\mathrm{LiveObs}(a,t)$ when authenticated administrative status is available. Current authority needs live status for continuing and temporary instruments, and for a one-shot instrument only under retroactive invalidation:

$$\begin{aligned}\mathrm{NeedsLive}(a) &\iff [\mathrm{mode}(a) \neq \text{one-shot}] \vee \mathrm{Retro}(a),\\ \mathrm{AppObs}(a,t) &\iff \mathrm{ProofObs}(a,t) \wedge\\ &\qquad [\mathrm{NeedsLive}(a) \Rightarrow \mathrm{LiveObs}(a,t)].\end{aligned}$$

Write V_1 for validity at execution without an effective retroactive invalidation, V_c for current validity without revocation, and V_e for V_c while the declared exception has not expired. The three-valued *current-authority* function is

$$\mathrm{Applicable}(a,\tau,t) = \begin{cases} \bot, & \neg\mathrm{AppObs}(a,t);\\ V_1(a,\tau,t), & \text{one-shot};\\ V_c(a,t), & \text{continuing};\\ V_e(a,t), & \text{temporary exception}.\end{cases}$$

This function is used by d_{auth}. A retained historical proof is therefore not confused with currently effective authority. Missing proof or required live status yields undecidable; an authenticated expiry or revocation yields a polar current-authorization finding.

Intent asks a different question: whether the revision now driving delivery belongs to a lineage that was authorized when each transition executed. Let $V_\tau(a,\tau)$ be the execution-time validity established by the immutable proof, let $R_{\mathrm{retro}}(a,t)$ be authenticated status saying that the execution has been retroactively invalidated (and structurally false when $\neg\mathrm{Retro}(a)$), and define

$$\begin{aligned}\mathrm{ExecObs}(a,t) &\iff \mathrm{ProofObs}(a,t) \wedge\\ &\qquad [\mathrm{Retro}(a) \Rightarrow \mathrm{LiveObs}(a,t)],\\ \mathrm{ExecApplicable}(a,\tau,t) &\equiv \mathsf{EA}(a,\tau,t),\\ \mathsf{EA}(a,\tau,t) &= \begin{cases} \bot, & \neg\mathrm{ExecObs}(a,t);\\ V_\tau(a,\tau) \wedge & \\ \neg R_{\mathrm{retro}}(a,t), & \text{else},\end{cases}\\ \mathrm{IntentApplicable}(a,\tau,t) &\equiv \mathsf{EA}(a,\tau,t).\end{aligned}$$

Thus a later prospective expiry or revocation can invalidate current authority while leaving execution-time lineage valid. Loss of prospective live status can likewise make current authority undecidable without erasing historically authorized intent. Live status enters the intent relation only when the declared policy permits retroactive invalidation; in that case its loss makes intent undecidable rather than silently preserving lineage.

Legitimate successors. Let $\mathscr{P}_{B,\iota}(t)$ be the finite set of candidate paths from ι_B to ι in the integrity-valid transition graph available at t. Each edge $e_j = (\iota_{j-1}, \iota_j, \delta_j^{\mathrm{exp}})$ carries its execution time τ_j and authorization instrument a_j; its expected artifact digest is bound to the approval/intent record. In strong three-valued logic, define

$$\begin{aligned}L(p,t) &= \bigwedge_{e_j \in p} \big[\mathrm{Covers}(a_j,e_j) \wedge\\ &\qquad \mathrm{IntentApplicable}(a_j,\tau_j,t)\big],\\ \mathrm{Succ}(\iota_B,\iota,t) &= \bigvee_{p \in \mathscr{P}_{B,\iota}(t)} L(p,t).\end{aligned}$$

The disjunction is evaluated only when completeness of the relevant transition graph is established; otherwise $\mathrm{Succ} = \bot$. On a complete graph it is false only when every candidate path is false. Under strong Kleene semantics, unknown execution applicability contributes $\bot$ to a path unless another conjunct already refutes that path; $\mathrm{Succ} = \bot$ exactly when no path is true and at least one path remains $\bot$. A missing graph segment, coverage or execution proof, or retroactive-status item is therefore never silently converted into a negative historical-authorization fact. A revision is a legitimate successor exactly when Succ is true. This excludes mere Git ancestry: lineage without a valid approval edge does not extend the admitted basis. The edge digest δ_j^{exp} is record-side evidence available at T2; it is not the executing digest δ_t. Comparing δ_t with covered expected digests remains the T3 responsibility of d_{auth}.

The basis revision itself also needs an execution-time lineage proof rather than an equality shortcut. Let $\mathscr{M}_B^{\mathrm{int}}$ be the complete family of inclusion-minimal instrument sets in A_B^{proof} that establish the basis revision at activation time τ_B. Define

$$\mathrm{BasisIntent}(B_x,t) = \bigvee_{M \in \mathscr{M}_B^{\mathrm{int}}} \bigwedge_{a \in M} \mathrm{IntentApplicable}(a,\tau_B,t),$$

$$\mathrm{IntentCovered}(\iota_B,\iota,t) = \begin{cases} \mathrm{BasisIntent}(B_x,t), & \iota = \iota_B;\\ \mathrm{Succ}(\iota_B,\iota,t), & \iota \neq \iota_B.\end{cases}$$

If the relevant minimal-set family or its completeness proof is unavailable, BasisIntent $= \perp$. This keeps the base case subject to declared retroactive invalidation while preventing a later prospective administrative change from rewriting historically authorized lineage.

B. The Governance-Divergence Vector

We define one divergence component per substantive relation; each is a function of exactly the inputs its component needs, which is what makes the detectability results of Section VI possible to state.

- $d_{\text{conf}}(t)$: a normalized difference over the managed configuration graph between desired(t) and m_t (field-level graph diff with runtime-owned fields factored out; Section VI shows the normalization is not cosmetic but the difference between signal and noise). This is an operational consistency component included to give the evaluator a complete control-state report; nonzero d_{conf} alone does not imply that either the desired or observed state lacks authorization coverage.
- $d_{\text{pol}}(t)$: the set of controls in $\mathscr{P}(t)$ that the running state violates although $B_x(t)$ records compliance under π_B—empty iff admission would re-succeed under today's policy. Reported as a violated-control set, not a number: policy versions form a revision structure, not a metric space, and "how far" is policy-weighted judgment, not geometry.
- $d_{\text{auth}}(t)$: the set of relied-upon instruments for which Applicable(a,τ,t) is false, plus running digests in δ_t not covered by an applicable instrument grounded in A_B^{proof} and $\mathscr{A}_{\text{live}}(t)$. If applicability is $\perp$, the authorization mask entry is zero and the substantive component is undecidable rather than included in this set.
- $d_{\text{int}}(t)$: let ι_t be the revision driving delivery in $\mathscr{I}(t)$. The component is zero when IntentCovered(ι_B,ι_t,t) is true and nonzero when it is false. When it is $\perp$, d_{int} has no substantive value and the corresponding intent mask entry is zero. Because this predicate uses execution-time lineage, loss of live status for a merely prospective continuing authority does not mask intent; missing execution proof or status required by a retroactive policy does.
- $d_{\text{env}}(t) = \{q \in \sigma_B : q(\varepsilon_t) = \text{false}\}$. The laboratory's declared IAM-scope and exposure fields instantiate equality predicates, but the definition permits ranges, membership, and monotone safety predicates.

The distinction is observable in the compound cases. Removing prospective live status while retaining the basis execution proof (S9) makes current authorization undecidable but leaves intent decidable and consistent. An uncovered rollback combined with that same loss (S12) leaves intent polar and inconsistent while authorization is undecidable. The report preserves both facts; evidence failure in one component cannot erase a decidable substantive finding in another.

Evidence availability is ontologically different from those distances. Let $\mathsf{EContract}_{\mu,j}(u,\theta;T)$ mean that every item required by component j in mode μ is present, complete, fresh within its declared bound, integrity-valid, source-authenticated, linked to u, and delivered by a healthy stream. Define

$$o_{\mu,j}(u,\theta;T)=1 \iff \mathsf{EContract}_{\mu,j}(u,\theta;T) \wedge \mathrm{ModeCut}_{\mu,j}(u,\theta;T),$$

$$\mathsf{O}_\mu(u,\theta;T) = \big(o_{\mu,\text{conf}}, o_{\mu,\text{pol}}, o_{\mu,\text{auth}}, o_{\mu,\text{int}}, o_{\mu,\text{env}}\big)(u,\theta;T).$$

Confidence is therefore not an unscoped probability: it is a named contract, mode, cut, and threshold published with the evaluator. A missing or contract-failing record yields *undecidable*; an authenticated record stating "revoked," "negative," or "never issued" is available evidence and may yield *inconsistent*. Evidence drift is a transition of a required mask entry from one to zero. Each substantive component therefore has three-valued reporting semantics—consistent, inconsistent, or undecidable—but only the first two are values of the substantive relation.

Definition 9 (Governance Divergence Vector; Consistency)*: For* $J = \{\text{conf},\text{pol},\text{auth},\text{int},\text{env}\}$*, let*

$$\mathsf{GD}_{\text{sub}}(u,\theta) = \big(d_{\text{conf}}, d_{\text{pol}}, d_{\text{auth}}, d_{\text{int}}, d_{\text{env}}\big)(u,\theta),$$
$$\mathsf{GD}_\mu(u,\theta;T) = \big(\mathsf{GD}_{\text{sub}}(u,\theta), \mathsf{O}_\mu(u,\theta;T)\big).$$

At fixed $(u,\theta;T)$*, suppress shared arguments and define*

$$\mathsf{OpCons}_\mu \iff o_{\mu,\text{conf}} = 1 \wedge d_{\text{conf}}(u,\theta) = 0,$$
$$\mathsf{GovPlaneCons}_\mu \iff \bigwedge_{j \in J\setminus\{\text{conf}\}} [o_{\mu,j} = 1 \wedge d_j(u,\theta) = 0].$$

Let D_μ *collect known nonzero components and* U_μ *collect components whose evidence contract fails:*

$$\mathsf{D}_\mu(u,\theta;T) = \{j \in J : o_{\mu,j} = 1,\ d_j(u,\theta) \neq 0\},$$
$$\mathsf{U}_\mu(u,\theta;T) = \{j \in J : o_{\mu,j} = 0\}.$$

Suppressing those shared arguments in the case conditions, the total status is

$$\mathsf{Status}_\mu(u,\theta;T) = \begin{cases} \text{inconsistent}, & \mathsf{D}_\mu \neq \varnothing; \\ \text{undecidable}, & \mathsf{D}_\mu = \varnothing \wedge \mathsf{U}_\mu \neq \varnothing; \\ \text{consistent}, & \mathsf{D}_\mu = \mathsf{U}_\mu = \varnothing. \end{cases}$$

These cases are exhaustive, and precedence of a known nonzero component means an incomplete vector may be substantively inconsistent while retaining zero mask entries for what remains unknown. Write $\mathsf{Consistent}_\mu(u,\theta;T)$ *iff this total status is* consistent*;* $\neg\mathsf{Consistent}_\mu$ *means "not established consistent" and is* not *a synonym for substantive inconsistency.*

Onsets are episode- and class-indexed. Let α_k *be the activation, re-approval, or verified recovery event that opens episode* k*, and let* $d_j^\star(u,t)$ *denote the underlying substantive relation*

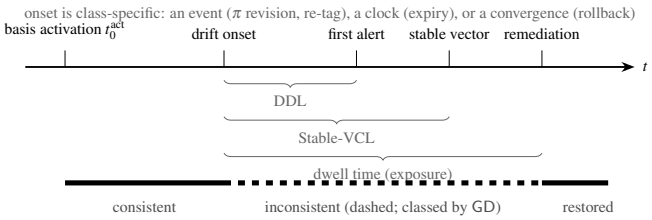

Fig. 4. The drift lifecycle and its temporal quantities (Definition 9). Substantive onset is episode- and class-specific—an organizational event (policy revision), a clock crossing (exception expiry), an exogenous event (registry re-tag), or reconciler convergence (rollback); evidence onset is the distinct transition of a required mask entry to zero. Drift detection latency (DDL; Metric M2) ends at the first honest alert; stable vector-completion latency (Stable-VCL; Metric M2) ends only when an admissible cut is closed by the required watermarks. Without that closure Stable-VCL is censored. Substantive and evidence dwell are reported separately; their remediation families are also distinct (Section IV-C).

independent of whether it is observable. For the declared evaluation track, abbreviate its logical-cut schedule by $\theta_k(t)$ and its time-varying mask entry by $o^k_{\mu,j}(t) = o_{\mu,j}(u, \theta_k(t); t)$. Then

$$t^{\text{sub}}_{k,j} = \inf\{t > \alpha_k : d^\star_j(u,t) \neq 0\},$$
$$t^{\text{evd}}_{k,j} = \inf\{t > \alpha_k : o^k_{\mu,j}(t^-) = 1 \wedge o^k_{\mu,j}(t) = 0\}.$$

An empty onset set has infimum $+\infty$; if an episode opens with the relation already nonzero or its evidence already masked, the corresponding onset is left-censored at α_k. Substantive dwell is the duration for which $d^\star_j \neq 0$ in that episode; evidence-failure dwell is the duration for which the required mask entry remains zero. Detection latency is detection time minus the explicitly declared relevant onset. The true substantive onset can be interval-censored when evidence arrives late; that uncertainty is reported rather than replaced by the evidence-failure time.

C. Why the Distance Is a Vector, Not a Scalar

A scalar GD was the natural first proposal, and we reject it as primitive for three reasons. *Incommensurability*: the components take values in different structures (graph diffs, control sets, validity states, three-valued verdicts); any scalarization imports policy weights, and those weights are governance choices, not properties of the system. *Remediation asymmetry*: the components have different remediations—reconcile (d_{conf}), re-evaluate and re-approve or migrate (d_{pol}), re-authorize or terminate the exception (d_{auth}), restore or re-approve intent (d_{int}), restore assumptions or re-assess (d_{env}), repair evidence where an entry of $\mathsf{O}_\mu(u, \theta; T)$ is zero—so collapsing them destroys exactly the information an operator needs. *Temporal structure*: components drift on different clocks (policy revisions are discrete organizational events; exception expiry is a scheduled instant; registry re-tags are exogenous), so a scalar time series conflates processes that must be attributed separately. A weighted composite for reporting is legitimate *on top of* the vector (Section VIII), with published weights, exactly as a risk score summarizes but does not replace findings.

D. Detectability, Formally

Each component divergence is a function of specific streams: d_{conf} of (desired, G_{obs}); d_{pol} additionally of $\mathscr{P}(t)$ and π_B; d_{auth} of A_B^{proof}, $\mathscr{A}_{\text{live}}(t)$, execution times, current Applicable status, and the actually executing digests δ_t. In contrast, d_{int} is a function of $\mathscr{I}(t)$, ι_B, the complete integrity-valid transition graph, its edge-bound expected digests, immutable execution proofs, and transition execution times; it reads live status only for instruments whose declared policy permits retroactive invalidation and never reads δ_t; d_{env} of ε_t and σ_B; and every entry of $\mathsf{O}_\mu(u, \theta; T)$ of the evidence and mode-specific cut needed for its corresponding computation. Architecturally, T2 owns the transition graph, edge-bound expected digests, and proofs needed to decide ExecApplicable; T3 adds the digest actually executing and is the first tier that can compare it with those expected digests. Proposition 1 is the negative direction of this dependency structure; the positive direction is equally direct:

Proposition 3 (Tier sufficiency)*: A detector with access to the streams a component depends on, an unambiguous unit-reference join, and* $\text{ModeCut}_{\mu,j}(u, \theta; T)$ *for its declared mode decides that component's consistency (in the closed-world model, exactly; in the field, up to the fidelity of those contracts). If either the unit join or declared cut fails, the sound result is undecidable. Consequently a sufficient detector for full governance consistency joins the configuration comparison with the policy, authorization, intent/lineage, and environment streams—the tiered architecture of Section V, whose scenario-to-tier realization Section VI checks cell by cell.*

Proof. Each component divergence in Section IV-B is a computable function of the named streams at one unit and cut. Providing those streams under the unit-reference, evidence, and declared-mode cut contracts provides the function's inputs; failure of any required contract is represented by the corresponding zero mask entry. □

The proposition is an input-enumeration result; whether a concrete implementation realizes it is a separate, checkable question—Section VI performs exactly that check on our released reference semantics, and *only* that check: agreement there validates the implementation, not the model.

Proposition 4 (Tier minimality)*: For every stream added at T1–T4, there exist two worlds that are indistinguishable to every lower tier but have different governance verdicts for a component first decidable from that stream. Therefore no lower tier decides that component in general.*

Proof. Construct the pairs while holding all lower-tier observations fixed. For T1, keep desired and observed configuration identical and let the current policy admit the deployment in one world but reject it in the other. For T2, keep the T1 projection fixed and let the same effective exception be currently valid in one world but expired in the other, changing authorization. A separate T2 pair keeps all lower-tier state fixed but varies a complete, proof-bearing authorized transition edge

versus a complete graph that refutes such an edge, changing intent. Removing prospective live status while preserving execution proof changes only the authorization mask; intent remains decidable. For a declared retroactive policy, removing the required status can instead mask intent, consistently with ExecApplicable. For T3, keep the manifest tag and every T2 stream fixed but let the running tag resolve to an approved digest in one world and an uncovered digest in the other. For T4, keep all T3 streams fixed and change only an unmanaged environment property recorded in σ_B. Each pair has identical lower-tier projections and different component verdicts, so any lower-tier detector returns the same value in both and errs in one. □

Unlike Proposition 3, this is a necessity result: the staircase is not merely a convenient implementation decomposition. Its scope remains relative to the declared component semantics; adding a genuinely equivalent stream at a lower tier widens that tier's projection rather than refuting the result.

Two modeling notes bound the claims. First, *granularity*: whether the registry re-tag scenario is "configuration" drift depends on reference semantics—under tag semantics the configuration is unchanged; under digest semantics it changed. The model takes no side: it requires only that $B_x(t)$ record resolved digests, which relocates the question from metaphysics to evidence (what did the approval bind?), where Section VI shows it is decidable at the lineage tier. Second, *legitimate context change*: policy supersession does not make drift *culpable*—the deployment was legitimate under π_7 and the organization changed the rules. The model deliberately measures divergence, not blame; whether policy drift triggers migration, exemption, or re-approval is a governance decision the vector informs (Section X).

V. A Tiered Detection Architecture

Propositions 1 and 3 together prescribe the architecture: governance-drift detection is configuration comparison *plus* the minimal additional evidence streams, joined against the selected admitted basis. Figure 5 organizes it as cumulative tiers, each named for what it adds and each mapped to the drift classes it first decides.

A. Components

Approved-state recorder. At approval, append an immutable pending snapshot to G^x_{app}—manifest content, *resolved* digests, policy-version pin, immutable authorization proof with execution times, intent linkage, environment assumptions, scope, and predecessor edge. At operational effect, append or attest its activation; cancellation marks it aborted. Persist both events durably and independently of the delivery platforms. This is the architecture's one hard prerequisite: without a recorded approval basis there is nothing to drift *from*; where such records are absent or decayed, the basis-dependent entries of $\mathsf{O}_\mu(u,\theta;T)$ are zero and those components are undecidable. If no independently evaluated component is known nonzero, the total status is undecidable; if configuration or another independently established component is known nonzero, the total status is inconsistent because known inconsistency precedes unknown evidence. The desired/observed configuration component remains comparable when its own evidence contract is satisfied; the evaluator must report this partial vector and mask rather than either total consistency or total ignorance. Conflicting activated maxima also fail closed instead of being ordered by approval timestamp (Proposition 2).

Evidence tiers. *T0/T0n*: the desired/observed comparison every reconciler already performs [3, 4, 6]; T0n normalizes runtime-owned fields (replica counts under autoscaling, rollout hashes)—the study quantifies why the naive form is unusable as a governance signal. *T1*: the policy-version stream from the policy engine's repository [9, 10]; the evaluator re-evaluates the running state against $\mathscr{P}(t)$ and compares with the pinned basis. *T2*: authorization records with mode and validity semantics—expiry, scope, prospective or retroactive revocation—bound to the expected change graph, covered digests where the authorization scope requires them, and immutable approval/exception proofs. Exceptions and approvals are therefore evidenced objects with clocks, subjects, and coverage rather than booleans; ITIL-style change records [18] and platform approvals both map here. *T3*: artifact lineage—digest-level linkage from running workloads to approval subjects, the slice pioneered by supply-chain attestation [19, 20, 23, 11]. *T4*: environment inventory—observation of the unmanaged properties assumed at approval (IAM scope, exposure) and comparison against the *recorded* predicates σ_B in $B_x(t)$, from cloud inventories, audit streams, and telemetry correlation [24, 13, 25]. Mechanically this is baseline comparison—CSPM's move—and we say so plainly; the difference is the reference (the approval event's recorded assumptions rather than a benchmark) and the joined, class-resolved verdict.

Evaluator. A continuous process (or scheduled sweep) computing $\mathsf{GD}_\mu(u,\theta;T)$ per deployment: cheap incremental checks on context-change events (policy revision published, exception clock elapsed, registry event), using event-triggered recomputation where a source exposes deltas and scheduled or background re-evaluation where it does not. Several scenarios in Section VI are detectable with zero latency at the tier that carries the relevant modeled event; this is not a claim that live integrations avoid polling or scans.

Analytic scaling envelope. For n deployments with at most m managed configuration fields, r policy predicates, a relied-upon instruments, d running digests, and q environment predicates each, a full sweep is $O(n(m+r+a+d+q))$ time. Retaining s complete snapshots per deployment costs $O(ns(m+a+d+q))$ storage, plus current-stream indices. The intended execution is incremental: policy-to-deployment, subject-to-approval, digest-to-workload, and predicate-to-resource indices restrict an event to its k affected deployments, yielding $O(k(m+r+a+d+q))$ work. Policy results and immutable proofs are cacheable; evaluators can shard by cluster, namespace, or tenant. Kubernetes watches and batched cloud-inventory deltas avoid one API request per deployment. Cloud

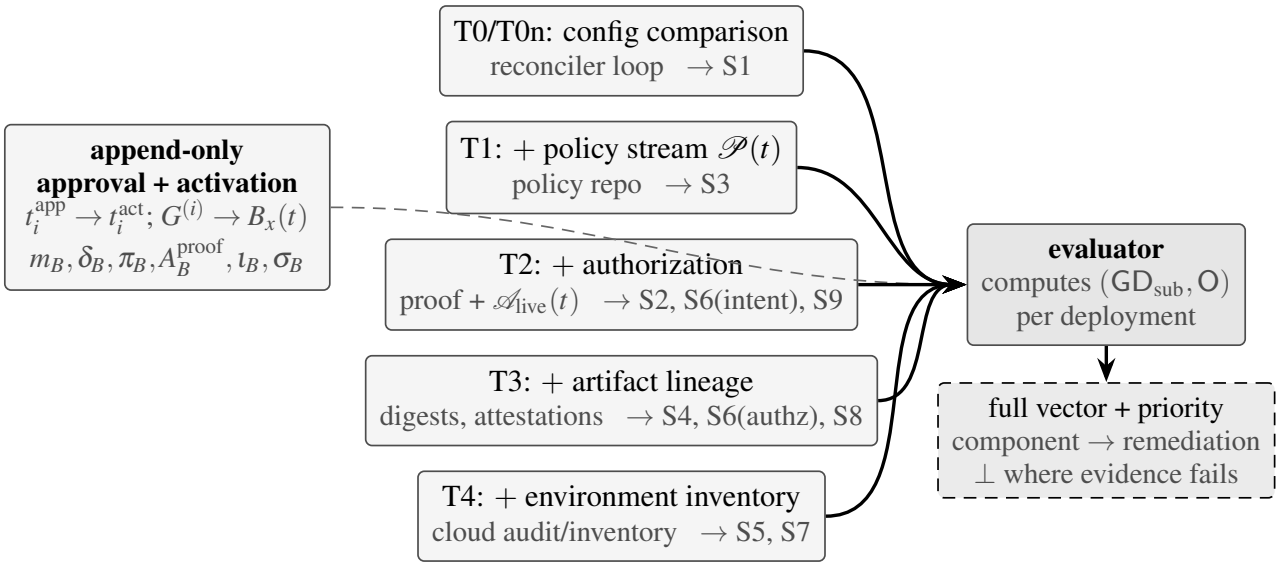

Fig. 5. Tiered governance-drift detection. The append-only recorder (left) preserves successive approvals and selects $B_x(t)$. Evidence tiers (center) cumulatively widen the detector's vocabulary: T0/T0n configuration comparison (the reconciler's existing loop, naive or churn-normalized), T1 policy-version stream, T2 authorization records with validity semantics, T3 artifact lineage, T4 environment inventory. The evaluator (right) recomputes the component divergences of Section IV-B continuously and emits classed drift vectors plus an epistemic mask, with a priority projection and the remediation family each component implies. Stream adapters also report the presence, completeness, freshness, integrity, authenticity, subject linkage, and health required by each component's evidence contract. Annotations show which components of Scenarios S1–S9 each tier first detects, as measured in Section VI.

quotas and cache invalidation remain engineering constraints. The in-memory microbenchmark in Table XIII measures the join after decoding and indexing inputs; it deliberately does not claim a Kubernetes or production throughput bound.

Minimal adoption path. The smallest useful deployment is T0n+T1+T2: normalize the reconciler comparison, re-evaluate running state under current policy, and join durable approvals and exceptions with explicit validity semantics. This separates operational, compliance, and authorization risk immediately; T3 adds digest lineage and T4 adds declared environment predicates as evidence maturity grows. Governance Conformance Coverage (GCC; Metric M6) is reported from the first rollout so unevaluable coverage is visible rather than treated as conformance.

For legacy workloads, a discovery pass may create a signed *retrospective baseline* from current state, current policy results, and an accountable owner's attestation. It is timestamped and labeled as reconstructed evidence—never represented as a historical approval—and GCC remains partial for facts that cannot be recovered. New changes then enter the ordinary append-only path, allowing coverage to increase without rewriting the estate's past.

B. Threat and Trust Model

The protected claim is integrity and decidability of the relation, not availability of the workload. An attacker may mutate Git, tags, runtime objects, policy results, authorization status, inventory, or one evidence adapter; operators may also make the same changes accidentally. These sources are therefore evidence inputs, not intrinsically trusted truth. The trusted computing base (TCB) is the recorder/activation authority, stable-identity issuer, authenticated time and adapter metadata, append-only log or external anchor, basis selector, and evaluator code. Every record is subject-bound, sequenced, freshness-bounded, and content-addressed; snapshots are complete atomic tuples chained to their predecessors. A contract failure masks only the dependent components and prevents a conformance claim.

A local hash chain detects modification and reordering of retained snapshots, but cannot by itself prove that a privileged actor did not truncate the tail or replace the entire history. Production deployment therefore requires an external transparency anchor or threshold signatures, access separation, and key/clock rotation records. Compromise of the evaluator and all anchors, or collusion of the recorder and activation authority, is outside the model. Denial of evidence remains visible as reduced GCC/undecidable; remediation may keep a critical workload available, but it must not relabel uncertainty as conformance (Section X).

C. Relation to Existing Mechanisms

The architecture is deliberately assembled from streams that already exist in mature platforms; its novelty is the *join semantics* against a recorded approval basis and the class-resolved verdict. Admission control [8, 9, 10] evaluates the same predicates once, at entry; continuous-validation features [11] re-evaluate the artifact-policy slice; posture management [13] scans environment configuration against benchmark policy without any approved-state reference. Indeed, per-tier continuous detection largely *exists* in cited tooling, and the architecture credits it explicitly: Kyverno's background scanning re-evaluates existing resources against current policy (our T1) [10]; configuration

recorders track unmanaged resources (much of T4) [6, 13]; continuous validation re-checks artifact policy (part of T3) [11]. Among the mechanisms in the bounded source set documented in Section IX, none jointly provides all of the following: every tier evaluated against the *admitted basis* $B_x(t)$, with mode-aware validity on the authorization itself, and verdicts that distinguish "violates policy" from "was never approved" from "was admitted legitimately and the world moved"—the distinctions that determine remediation (Section IV-C). The contribution is the relation and its join semantics, not the comparison machinery, and Section IX restates novelty in exactly those terms.

VI. Executed Study: Detectability Across Evidence Tiers (RQ2–RQ4)

We now measure the taxonomy's detectability structure. The study is executed and deterministically reproducible from the packaged dependency-free Python source; the results table, including its header, is generated programmatically by the same run. Its epistemic scope is stated plainly: it is a *closed-world evaluation of detector semantics*—the world, its scenarios, and the detectors are all models—so it establishes what each evidence tier can and cannot decide *in principle*, with event-time latencies. It does not measure any real cluster; wall-clock behavior on real infrastructure is the laboratory's job (Section VII).

A. Modeled World and Scenarios

The world models one deployment with the selected admitted basis of Section IV-A—including recorded environment assumptions σ_B—and streams for desired state (Git), observed cluster state, registry contents, policy versions *with evaluable requirement sets* (so that T1 implements Definition 3's admission re-evaluation, not a version-string comparison), authorization records as a set with subject digests, covered revisions, windows, and revocation (so that legitimate successors are modeled), and two unmanaged environment properties. Each run is 400 events; drift is injected at event 150. Twelve scenarios follow the taxonomy and the constructed examples of Figure 3:

S1 manual in-cluster change (observed field diverges from desired);

S2 expired emergency authorization (exception granted with a 40-event window; nothing removes it at expiry);

S3 policy version change post-deploy ($\pi_7 \to \pi_8$, which the running configuration violates);

S4 artifact substitution (same tag re-pointed to a new digest, rolled onto nodes; manifests unchanged);

S5 IAM permission expansion out of band (unmanaged);

S6 Git rollback without new approval (reconciler converges to the reverted content; approval binds the newer revision);

S7 out-of-band cloud console modification (unmanaged);

S8 approval mismatch (deployed digest is not any approval's subject);

S9 live-status deletion for a continuing approval: immutable A_B^{proof} survives, but the authenticated source required to establish continuing validity is lost; current authorization becomes undecidable, while historically authorized intent remains decidable and consistent—the missing status is neither a revocation nor proof that approval never existed (evidence drift—Definition 6—exercised directly);

S10 policy supersession plus an expired exception ($\{\text{policy}, \text{authorization}\}$);

S11 artifact substitution plus environment change ($\{\text{authorization}, \text{environment}\}$);

S12 rollback plus loss of the authenticated live status required by continuing authority, while immutable proof remains: the rollback is outside the authorized historical lineage, so intent is inconsistent; current authorization is independently undecidable. The honest class set is therefore $\{\text{intent}, \text{evidence}\}$ rather than either a guessed authorization pole or evidence alone;

plus **S0**, a no-drift control. Churn is of two kinds, both present in every stream including the control: *runtime churn* (30% of events: autoscaler replica changes, rolling-restart hash changes) and *benign governance churn* (2% of events: policy revisions the running state satisfies, approved updates that legitimately advance revision, digest, and approval together, and hygienic exceptions removed at expiry). The governance churn exists so that the governance tiers' false-alarm rates are earned against modeled non-silent governance churn rather than measured against a silent plane.

B. Detectors and Counterfactual Scoring

Six detector tiers implement Section V: T0 (naive configuration comparison, including runtime-owned fields), T0n (normalized), and cumulative T1–T4, each a pure function of its tier's visible streams, so tier capabilities are enforced by construction. Ground truth is a *class set* per scenario (components are a vector and may drift together, Definition 8). The detector exports the complete class set decidable at its tier; a priority-ordered first verdict is derived only as an operational interface. We score exact-set agreement, subset-only partial diagnosis, and Hamming loss rather than accepting any one member of the ground truth. Missing approval records set the relevant entries of O to zero and produce the explicit undecidable/evidence verdict, never a guessed polar one. S6, S10, and S11 test substantive composition; S12 tests composition under evidence loss.

Scoring required care, and the naive arm is why. A detector that alarms constantly "detects" every scenario by coincidence; volume is not information. We therefore score *counterfactually*: for each (scenario, tier, seed), the detector runs on the drift stream and on a paired, same-seed, no-drift control stream with identical churn; the drift is *detected* iff the two alarm sequences

differ, detection time is the first differing event, and classification is the class set emitted there. Under this rule a constant alarmer carries zero information and scores zero. False alarms are counted on the control stream. Twenty seeds per cell; onset for S2 is the expiry instant (the drift is the exception *outliving* its window, not its grant).

C. Results

Table II is the study's central artifact. Four findings, stated in the register the design supports.

Finding F1 (The staircase: implementation validation of tier dependencies)*: Detection forms a strict staircase realizing Propositions 3 and 4 cell for cell: T0n detects S1 and nothing else—one scenario of twelve; T1 adds policy drift (S3) and the policy component of S10, and—because it re-evaluates policy rather than comparing versions—raises no alarm on satisfied policy revisions in the churn. T2 adds the validity- and coverage-dependent scenarios S2, S6, and S10, plus S9's authorization-evidence loss and S12's compound polar-intent and epistemic result. T3 adds the digest-level scenarios S4, S8, and S11 and completes S6's two-component diagnosis; T4 adds S5/S7 and completes S11. The triangle cells show why first-label accuracy was insufficient: a lower tier can report a correct subset without deciding the full vector. Every reported component is correct in every seed, and exact-set agreement appears when the tier carries all required streams. We state this finding's epistemic value precisely: the world and detectors are co-designed, so the exactness is expected, and the staircase is* implementation validation *of the model's dependency structure plus a noise-behavior measurement—not field evidence for the taxonomy, a role we do not claim for it. Its concrete content for RQ4 stands: in the reference semantics, configuration comparison decides only its own substantive component; the five-class substantive vector plus epistemic mask requires the joined tiers. All six umbrella classes are exercised.*

Finding F2 (A deliberately naive comparator validates the scoring rule)*: The unnormalized diagnostic comparator T0 alarms on 285.4 of 400 control events (71.3% in our churn model)—and the rate is nearly flat across runtime churn rates of 10–50% (286.5, 285.4, and 300.1 mean alarms per run), because alarms track the* dwell *of divergent runtime-owned state, not the event rate. Under counterfactual scoring T0 still detects S1, but with mean latency 22.6 events versus 0 for T0n: its information is buried in its own noise. For the other eleven scenarios it carries zero information while alarming continuously. T0 is not an industry baseline—production reconcilers normalize—and no comparative effectiveness claim rests on it; its sole role is to demonstrate that the counterfactual scoring rule does not reward alarm volume. Conversely, the governance tiers' zero false alarms are now* earned*: the control contains satisfied policy revisions (which a version-comparison T1 would have flagged), legitimate approved updates (which a pinned-revision intent check would have flagged), and hygienic exceptions—and the definition-faithful detectors correctly ignore all of them.*

Finding F3 (Governance drift is event-detectable, with one designed exception)*: At the correct tier, every scenario is detected with latency 0 events (including S2 under expiry-inclusive semantics). In this closed-world event model, no additional intrinsic delay is introduced once the deciding fact is available at the appropriate tier. Live visibility remains bounded by the concrete re-evaluation path—watch, cache, controller audit, scan, and evaluator cadence—as the Kyverno results in Section VII demonstrate. RQ3's closed-world answer is therefore about information availability, not a claim that real policy re-evaluation is scan-free.*

Finding F4 (Reconciliation health is compatible with governance inconsistency)*: In eleven of twelve scenarios, post-convergence normalized configuration comparison reports clean—while the deployment runs under superseded policy, expired authorization, unapproved content, or altered environment. This is Proposition 1 made operational, and it quantifies the blind spot claimed in Section II: a* SYNCED *reconciler is evidence about desired/observed equality within the reconciler's managed projection.*

D. Separately Implemented Composite Baseline: Signals Without the Join

The staircase establishes dependency realization but does not isolate the value of the admitted-basis join. We therefore execute a second baseline that unions five plane-local mechanisms: normalized desired/observed comparison, current-policy evaluation, exception-expiry checking, digest membership in a tool-local attestation trust set, and current environment posture against a benchmark. It receives the same event stream and benign churn as T4 but has no $B_x(t)$, snapshot selection, intent history, or evidence-state semantics. This is a stronger baseline than T0 and represents composition of existing signals without claiming to reproduce a particular commercial product.

The local union exactly identifies 9/12 scenario sets (180/240 scenario-seed units, 75%); joined T4 identifies 12/12 (240/240, 100%). Both remain quiet on the paired controls, so the difference is diagnosis, not alarm volume. The local union misses S6's intent component, cannot see S9's evidence loss, and labels S12 as authorization while missing both the decidable intent finding and the honest authorization-evidence mask. To avoid attributing the entire difference to one monolithic step, we execute a paired cumulative ablation over the same 12-scenario $\times$ 20-seed units (Table IV). Ten executable semantic probes then target cases the scenario matrix does not separate: policy failure with/without a basis; continuing versus one-shot authority under missing live status; approved nonstandard environment; unapproved rollback; pending, aborted, parallel, and successor activation.

Exact-class-set success rises from 75.0% (B0) to 83.3% after evidence contracts (B1). Mode-aware authorization and epistemic masking (B2) preserves 83.3% while reducing mean Hamming loss from 0.0417 to 0.0278; it cannot add S12's intent component because intent history is deliberately absent. Adding that history (B3) raises exactness to 100%. B3 nev-

TABLE II
DETECTABILITY MATRIX, GENERATED PROGRAMMATICALLY FROM THE RUN OUTPUTS (20 SEEDS PER CELL). ✓ = DETECTED WITH EXACT-SET AGREEMENT IN ALL SEEDS; △ = DETECTED BUT ONLY A CORRECT SUBSET IS DECIDABLE AT THAT TIER (SUPERSCRIPT: MEAN DETECTION LATENCY IN EVENTS WHEN >0); × = NOT DETECTED (NO INFORMATION UNDER COUNTERFACTUAL SCORING). FP = FALSE ALARMS PER 400-EVENT CONTROL RUN.

Scenario	**T0** naive cfg	**T0n** norm. cfg	**T1** +policy	**T2** +authz	**T3** +lineage	**T4** +env
S0 control (churn only)	FP: 285.4	FP: 0.0	FP: 0.0	FP: 0.0	FP: 0.0	FP: 0.0
S1	✓$^{+22.6}$	✓	✓	✓	✓	✓
S2	×	×	×	✓	✓	✓
S3	×	×	✓	✓	✓	✓
S4	×	×	×	×	✓	✓
S5	×	×	×	×	×	✓
S6	×	×	×	△	✓	✓
S7	×	×	×	×	×	✓
S8	×	×	×	×	✓	✓
S9	×	×	×	✓	✓	✓
S10	×	×	△	✓	✓	✓
S11	×	×	×	×	△	✓
S12	×	×	×	✓	✓	✓

S1 manual change; S2 expired exception; S3 policy supersession; S4 artifact substitution; S5 IAM expansion; S6 unapproved rollback; S7 out-of-band cloud change; S8 approval mismatch; S9 required live-status evidence loss; S10 policy+exception; S11 artifact+environment; S12 rollback+required authorization-evidence loss. Tiers are cumulative; T0n normalizes runtime-owned fields; latencies use expiry-inclusive semantics.

ertheless passes only 5/10 targeted probes; activation-aware basis selection (B4) preserves 100% exact scenario class sets and raises probe coverage to 10/10. This separation is important: the scenario score alone would have hidden pending/aborted bases, parallel-activation ambiguity, and approved nonstandard environments. Identical current-policy failure with versus without an admitted basis produces the same local "policy" alert, while the join returns policy versus evidence; missing live status yields evidence for a continuing instrument but remains consistent for a prospective one-shot instrument with retained proof. The incremental value is therefore the history- and mode-aware relation that selects both class and remediation family. As with the staircase, this closed-world comparison validates semantics, not field effectiveness.

E. Threats to Validity

Construct: detectors and world share a modeling vocabulary; the tiers' *exact* 0/1 staircase is a property of the closed world, where streams are noiseless and semantics are implemented as defined. Real streams are late, lossy, and ambiguous; the staircase's *shape* (which class needs which evidence) is the transferable claim—it follows from input dependencies (Proposition 3)—while the crispness is not. *Internal*: single-deployment world and one injection time per run. Three compound scenarios exercise set output, but do not span the combinatorial space. The first-verdict order is one operational policy among several; exact-set scoring is invariant to it. Scenario S6's transient configuration signal (before the reconciler converges) is not credited to T0n because the comparison evaluates post-convergence state in our event ordering; on real clusters a fast comparator might glimpse it—a cadence question for the lab. The no-join comparator and admitted-basis implementation use separate code paths, but both were authored by the same researcher and share the world, protocol, and oracle. "Separately implemented" therefore denotes code-path separation, not independent validation; the comparison is an ablation-style semantic comparator. *External*: the repeated primary laboratory (Section VII) shows that the nine scenario classes can be induced and usually observed through real Kubernetes/GitOps interfaces on one version-pinned stack. A bounded Argo CD/Gatekeeper campaign replicates native configuration and policy paths, but its authorization slice shares the digest adapter and it does not evaluate intent or environment. Neither campaign transfers the closed-world rates quantitatively or estimate natural occurrence, field prevalence, stream-loss reliability, multi-cluster behavior, or performance under load. Its two cadence misses are direct evidence against assuming that polling necessarily observes transient configuration drift.

VII. REPEATED KUBERNETES/GITOPS LABORATORY

The closed-world study fixes semantics; a live stack tests whether the declared classes and evidence dependencies are

TABLE III
SEPARATELY IMPLEMENTED PLANE-LOCAL BASELINE VERSUS THE ADMITTED-BASIS JOIN. EXACT COLUMNS ARE PERCENTAGES OVER 20 SEEDS; EMPTY LOCAL OUTPUT ON S9 MEANS THE DELETED GOVERNANCE STREAM IS OUTSIDE ITS VOCABULARY.

Scenario	Expected	Local union	Joined	Local exact	Joined exact
S1	$\{config.\}$	$\{config.\}$	$\{config.\}$	100	100
S2	$\{auth.\}$	$\{auth.\}$	$\{auth.\}$	100	100
S3	$\{policy\}$	$\{policy\}$	$\{policy\}$	100	100
S4	$\{auth.\}$	$\{auth.\}$	$\{auth.\}$	100	100
S5	$\{env.\}$	$\{env.\}$	$\{env.\}$	100	100
S6	$\{intent, auth.\}$	$\{auth.\}$	$\{intent, auth.\}$	0	100
S7	$\{env.\}$	$\{env.\}$	$\{env.\}$	100	100
S8	$\{auth.\}$	$\{auth.\}$	$\{auth.\}$	100	100
S9	$\{evidence\}$	$\{\}$	$\{evidence\}$	0	100
S10	$\{policy, auth.\}$	$\{policy, auth.\}$	$\{policy, auth.\}$	100	100
S11	$\{auth., env.\}$	$\{auth., env.\}$	$\{auth., env.\}$	100	100
S12	$\{intent, evidence\}$	$\{auth.\}$	$\{intent, evidence\}$	0	100

TABLE IV
CUMULATIVE ABLATION OF THE ADMITTED-BASIS JOIN. EXACT IS UNCONDITIONAL EXACT-CLASS-SET SUCCESS OVER PAIRED SCENARIO–SEED UNITS; PROBES ARE EXECUTABLE SEMANTIC CONTRACTS.

Variant	Added semantics	Units	Exact (%)	Hamming	Probes
B0	plane-local union	240	75.0	0.0694	3/10
B1	+ evidence contracts	240	83.3	0.0417	3/10
B2	+ mode-aware authorization and masking	240	83.3	0.0278	4/10
B3	+ intent history	240	100.0	0.0000	5/10
B4	+ activation-aware admitted-basis join	240	100.0	0.0000	10/10

concretely realizable through control-plane interfaces. We therefore executed a repeated laboratory over the nine atomic scenarios and six benign controls, followed by a smaller compound-class-set extension and six separate five-minute benign windows. A separate bounded campaign then exercised three slices on Argo CD and Gatekeeper. The purpose is deliberately bounded: realizability and within-laboratory detector behavior under the reported stacks. It is not evidence of natural occurrence, field prevalence, production effectiveness, or transfer across organizations.

A. Laboratory Hypotheses

The frozen protocol used for this campaign specifies three falsifiable hypotheses. **LH1 (realizability):** each injected scenario, when observed at or above its predicted minimum evidence tier, yields exactly the expected provisional class set under the laboratory's sequential-snapshot assumption. **LH2 (cadence sensitivity):** a transient drift interval can be missed when its dwell time is shorter than the evaluator cadence. **LH3 (transition safety within scope):** admitted-basis-preserving changes in the six benign families produce no policy, authorization, intent, or environment drift; genuine in-flight configuration divergence or temporarily incomplete evidence may be reported, but must recover after convergence. These are laboratory hypotheses, not prevalence or cross-platform claims.

B. Build and Reproducibility Boundary

A single-node Kind v0.32.0 cluster running Kubernetes v1.36.1 [26] hosts Flux v2.9.4 [4] and Kyverno v1.18.2 [10]. A local smart-HTTP Git service drives Flux and a local OCI registry supports tag/digest substitution. Approval records and the environment inventory are versioned JSON files, not production approval or cloud-inventory services. The current bootstrap verifies upstream manifests by SHA-256 and resolves images through a machine-readable digest lock. Because the frozen primary campaign records shared controller specifications by tag, controller-level digest pinning is not claimed for that run. Platform versions and the image references actually used are retained. An approved-state recorder snapshots G_{app}: manifest, resolved pod-image digest, policy pin, approval JSON with validity window, Git revision, and declared environment assumptions. The frozen primary campaign retains its original single-basis snapshot representation. Separately, the v1.6 B4 fixtures and current recorder separate approval from activation, record explicit supersession, and seal each complete snapshot with content and predecessor hashes. The selector verifies the chain and rejects absent, cyclic, or parallel maximal bases; targeted B4 probes cover pending, aborted, successor, and ambiguous parallel activation. These probes test the added selection semantics and are not observations from the frozen primary campaign. Tampering is detectable under the laboratory trust model, not impossible. The dependency-free evaluator reads Kubernetes, Git, Kyverno PolicyReports, file-backed live authorization status, pod image IDs, and the file-backed inventory through tier-restricted entry points T0n–T4. The bootstrap, injections, repeated harness, evaluator, raw observations, controls, and generated tables ship under `lab/`. The artifact additionally executes the full admissible-cut contract in a dependency-free selector: missing or lagging watermarks, stale evidence, capture intervals that straddle the cut, ambiguous latest records, and excessive cross-source spread all reject the cut. This tests the formal decision boundary separately; it neither upgrades the synchronous laboratory readers to production streams nor makes their sequential snapshots admissible formal cuts after the fact. The baseline deployment uses a continuing approval: its immutable proof is captured in $B_x(t)$, while the live JSON supplies current administrative status; S9 removes the latter and therefore makes continuing

validity undecidable. S9 consequently emits the transversal evidence class because the authorization component is undecidable; its historically authorized intent lineage remains decidable and consistent. In this bounded implementation, stream health and syntactic completeness are checked at read time; Kubernetes policy evidence is joined by namespace and object UID, while authorization records carry a deployment subject. Local files remain trusted sources, so independent source authentication is outside the primary campaign. The reference semantics are explicit: T0n compares the declared image reference (including its tag) as configuration, while T3 resolves the running `imageID` digest and tests whether an applicable approval covers that digest. Thus re-tagging can remain configuration-consistent yet fail lineage.

Measurements ran on an arm64 Apple M4 Max host with 48 GiB RAM and macOS 26.5.1, Docker 29.1.3, and Python 3.11.9. The Kind node reported 16 CPUs and 8,024,028 KiB memory; Docker imposed no explicit per-container CPU or memory limit. No benchmark workload ran concurrently, but ordinary host background services were neither disabled nor load-controlled. Durations use Python's monotonic clock (`mach_absolute_time()`, reported resolution 4.17×10^{-8} s), so wall-clock adjustment cannot invert intervals. These conditions make the reported seconds interpretable as measurements of this host, not portable performance guarantees; the machine-readable snapshot is stored with the observations.

C. Protocol and Five Timestamps

For each repetition, a seeded shuffle changes scenario order. Before every injection, the harness restores the approved Git revision, policy π_7, approval record, artifact tag, deployment, and inventory, and requires T4 to return a provisionally *consistent* classification under the laboratory snapshot assumption. Each S1–S9 scenario is executed 20 times. Three logical class-set observers sample the same injection at cadences 0.5, 2, and 10 s; their first post-onset phase is sampled independently and deterministically from $[0, c)$ for cadence c, after which they poll every c seconds. Thus a latency below 0.5 s is possible when the first scheduled poll falls shortly after onset; 0.5 s is a cadence, not a measurement-resolution floor. We report two decimal places and retain higher-resolution raw clocks only for recalculation. In the frozen positive campaign, the three schedules share one serial event loop. They are correlated logical observers, not isolated processes: a slow T4 call can delay another schedule. The result therefore supports cadence sensitivity under this harness, not independent observer performance. The transition campaign below removes that coupling with three processes and records scheduled time, actual start, completion, scheduler lag, and every verdict.

The experimental unit is the injected scenario repetition (180 units: 20 per scenario). The 540 cadence-specific observations are correlated repeated measurements over those same injections, not 540 independent experiments. For cadence-wise descriptive uncertainty only, we report Wilson intervals under the usual exchangeable-Bernoulli approximation. Dependence across cadences is handled by a seeded, scenario-stratified nonparametric bootstrap: injection clusters are resampled within each of nine scenario strata and the stratum means are macro-averaged. The estimand is detection over the balanced experimental mixture, not over a population of incidents.

The harness records five relevant timestamps: command initiation t_{inject}, operational inconsistency t_{onset}, first non-consistent class-set verdict t_{first}, first exact class-set completion t_{complete}, and completion of the minimal JSON evidence bundle t_{evidence}. It reports actuation, DDL, end-to-end latency, and TTE; the compound extension also reports ESC and diagnostic gap, exactly as defined in Metric M2. For command-carried mutations, onset is conservatively operationalized as successful command completion. For S2 it is exception expiry; for S4, successful rollout of the substituted artifact; and for S6, successful Flux convergence to the unapproved revision. Consequently, S6's Git commit and reconciliation belong to actuation and end-to-end time, not DDL. S3 begins at successful policy supersession, so its DDL includes waiting for Kyverno's background PolicyReport. Evidence assembly in this reference implementation is synchronous with verdict serialization, so TTE and DDL are nearly identical; this does not estimate retrieval delay for a platform-scale evidence service.

Ground truth is a set. S6 is $\{\text{intent}, \text{authorization}\}$; T2 can expose the intent component, while T3 lineage is required to decide its authorization component. Every live observation is therefore evaluated at T4 and compared as a complete class set; the recorded tier is the minimum predicted to decide that scenario. The priority-ordered first verdict is retained only as an operational interface. Detection rate (DR) counts a non-consistent or undecidable verdict before the 180-s timeout. Exact-set accuracy (ESA) requires equality of the complete observed and expected sets, conditional on detection; we also report per-component precision/recall and Hamming loss. Because the live evaluator reads its sources sequentially without authenticated watermarks, these are provisional classifications under a laboratory snapshot assumption, not evaluations of the formal Consistent predicate over an admissible distributed cut. The separate temporal-cut tests validate that contract independently.

D. Detection, Classification, and Cadence

At the default 0.5-s cadence, all 180 injections were detected and all detected provisional classifications exactly matched their expected class sets (Table V). The separation of clocks changes the interpretation materially. For S6, median actuation was 10.05 s, median DDL 0.54 s, and median end-to-end latency 10.50 s. The former one-pass value around 7 s was therefore not detector latency. Conversely, S3 had median actuation 0.13 s but median DDL 9.68 s (P95 10.38 s), exposing the background-policy path.

Across all cadences, 538 of 540 observations produced a verdict, and all 538 detected provisional class sets exactly matched their expected sets. Both misses were S1 in the same repetition: the 0.5-s evaluator observed the manual mutation, but Flux reconciled it before the first 2-s and 10-s polls. This is

TABLE V
REPEATED LIVE-STACK RESULTS AT THE DEFAULT 0.5-S CADENCE (20 RUNS PER SCENARIO). EXPECTED IS A CLASS SET; "FIRST VERDICT" IS THE EVALUATOR'S PRIORITY-ORDERED OUTPUT. DR AND ESA ARE PERCENTAGES. ACT., DDL, P95 DDL, END-TO-END (E2E), AND TTE ARE SECONDS; ALL EXCEPT P95 ARE MEDIANS.

ID	Expected set	Observed set	First	DR	ESA	Act.	DDL	P95	E2E	TTE
S1	{configuration}	{configuration}	configuration	100.0	100.0	0.05	0.45	0.65	0.50	0.45
S2	{auth.}	{auth.}	authorization	100.0	100.0	2.34	0.47	0.71	2.90	0.47
S3	{policy}	{policy}	policy	100.0	100.0	0.13	9.68	10.38	9.81	9.68
S4	{auth.}	{auth.}	authorization	100.0	100.0	1.56	0.53	0.68	2.10	0.53
S5	{environment}	{environment}	environment	100.0	100.0	0.03	0.44	0.58	0.46	0.44
S6	{intent, auth.}	{auth., intent}	intent	100.0	100.0	10.05	0.54	0.75	10.50	0.54
S7	{environment}	{environment}	environment	100.0	100.0	0.04	0.45	0.60	0.48	0.45
S8	{auth.}	{auth.}	authorization	100.0	100.0	0.02	0.40	0.67	0.44	0.40
S9	{evidence}	{evidence}	evidence	100.0	100.0	0.01	0.42	0.62	0.44	0.42

TABLE VI
CADENCE SENSITIVITY ACROSS NINE SCENARIOS AND 20 REPETITIONS. ESA IS CONDITIONAL ON DETECTION; DDL VALUES ARE SECONDS.

Cadence	Runs	DR (%)	ESA (%)	Median DDL	P95 DDL
0.5	180	100.0	100.0	0.48	10.08
2.0	180	99.4	100.0	1.45	10.47
10.0	180	99.4	100.0	6.25	14.61

not discarded as an outlier. It demonstrates the cadence limit predicted by **LH2**: polling a transient relation can miss it even when exact-set identification is perfect when observed.

At 0.5 s, detection was 180/180 (Wilson 95% CI 97.91–100%); at both 2 and 10 s it was 179/180 (96.92–99.90%). Scenario-stratified resampling of complete three-cadence injection clusters gives 99.63% macro-average detection over the balanced experimental mixture with a 95% percentile-bootstrap interval of 98.89–100% (50,000 resamples, seed 20260807). These intervals quantify repeated behavior in this protocol; they are not confidence intervals for production effectiveness. For a single transient of duration τ sampled with uniformly random phase at cadence c, the idealized miss probability is $\max(0, 1 - \tau/c)$; a production miss percentage additionally requires the field distribution of τ, which this study does not estimate.

Across all 540 cadence observations, the unconditional exact-classification success rate was 99.63%; conditional ESA remained 100% among the 538 detected observations. Unconditional Hamming loss was 0.000617. The two missed S1 labels account for all error: configuration recall was 96.7%; precision for every output and recall for policy, authorization, intent, evidence, and environment were 100% (Table VII).

Table VI reports the aggregate effect. DR was 100.0% at 0.5 s and 99.4% at both 2 and 10 s. Conditional ESA remained 100.0%. Median DDL rose from 0.48 to 1.45 and 6.25 s as cadence widened; P95 rose from 10.08 to 10.47 and 14.61 s. P95 is dominated partly by S3's PolicyReport delay rather than cadence alone, so these are distributions of the complete deciding paths on this laboratory, not calibrated evaluator-service times.

TABLE VII
CLASS-SET COMPONENT METRICS ACROSS ALL CADENCES. EVIDENCE IS THE EPISTEMIC OUTPUT TRIGGERED BY AN UNDECIDABLE SUBSTANTIVE COMPONENT, NOT A SIXTH SUBSTANTIVE DISTANCE.

Component	TP	FP	FN	Precision	Recall
configuration	58	0	2	100.0%	96.7%
policy	60	0	0	100.0%	100.0%
authorization	240	0	0	100.0%	100.0%
intent	60	0	0	100.0%	100.0%
evidence	60	0	0	100.0%	100.0%
environment	120	0	0	100.0%	100.0%

E. Process-Isolated Positive Trace Replication

The repeated campaign above uses correlated schedules in one serial event loop. We therefore executed a separate audit replication of S1–S9 in the exclusive `govdrift-trace` namespace. Each scenario episode starts three separate OS processes at 1-, 5-, and 10-s cadences. Every process emits an initial provisionally consistent pre-cause poll, then fsyncs every class-set observation to append-only NDJSON. Each poll contains its observer UUID and PID, scheduled and actual start, completion, scheduler lag, input fingerprint, and a SHA-256 link to the preceding poll. The corresponding injection record contains a unique ID, source hashes, before/after input fingerprints, command-ledger indices, and monotonic plus UTC timestamps for cause start and observed effect. From those records, the verifier independently reconstructs first alert, first exact class set, and two-poll exact persistence, defined solely as the second consecutive exact poll. This operational persistence check is not a watermark-qualified persistence claim.

The nine injected episodes yielded 27 correlated observer trajectories and 261 hash-chained polls. All 27 reached both the exact expected set and two-poll exact persistence, and no poll recorded an adapter exception (Table VIII). Cause-start-to-first-exact completion had median 4.465 s and nearest-rank P95 9.764 s; cause-start-to-two-poll exact persistence had median 9.522 s and nearest-rank P95 19.811 s. These are

TABLE VIII
PROCESS-ISOLATED POSITIVE TRACE REPLICATION. EPISODES ARE EXPERIMENTAL UNITS; THE THREE CADENCE TRAJECTORIES PER EPISODE ARE CORRELATED OBSERVATIONAL UNITS. EXACT AND 2-POLL ARE TRAJECTORY COUNTS.

Expected set (scenario)	Episodes	Traj.	Exact	2-Poll
Configuration (S1)	1	3	3	3
Policy (S3)	1	3	3	3
Authorization (S2,S4,S8)	3	9	9	9
Authorization+intent (S6)	1	3	3	3
Environment (S5,S7)	2	6	6	6
Evidence/undecidable (S9)	1	3	3	3
Total	9	27	27	27

TABLE IX
BENIGN LIVE-STACK CONTROLS. "POLLS" SUMS ALL THREE CADENCES; ALARM WINDOWS COUNTS WINDOWS WITH AT LEAST ONE SUBSTANTIVE DRIFT VERDICT; EPISTEMIC POLLS COUNT HONEST *Undecidable* OUTPUTS SEPARATELY.

ID	Benign control	Windows	Polls	Alarm windows	Epistemic polls
C1	satisfied policy revision	4	108	0	0
C2	approved rollback	4	108	0	0
C3	exception removed at expiry	3	84	0	0
C4	approved artifact re-tag	3	81	0	0
C5	autoscaling replica change	3	81	0	0
C6	legitimate rollout restart	3	81	0	0

pooled descriptive summaries over 27 correlated trajectories, not estimates from 27 independent experimental units and not DDL as defined above. The experimental units are the nine episodes. The campaign uses one local Kind cluster, explicitly triggered Flux reconciliation for S6, a Kyverno admission mutation to emulate S4, and file-backed cloud inventory for S5/S7; it strengthens positive traceability and reproducibility, not field prevalence, scalability, or production effectiveness. The frozen campaign, source snapshots, Git bundle, per-poll chains, result manifest, and cross-layer verifier ship under `lab/results_trace/`; the final campaign pointer excludes the retained pilot and intentionally aborted development runs.

F. Steady-State Benign Controls

The positive scenarios do not establish specificity. We first ran 20 10.5-s drift-free windows (210 s total), using a near-balanced round-robin allocation across six controls (4,4,3,3,3,3): satisfied policy revision (C1), rollback accompanied by approval for the new revision (C2), exception removal at expiry (C3), re-tag accompanied by approval for the new digest (C4), autoscaling replica change outside the managed projection (C5), and a legitimate rollout restart (C6). Every window was evaluated at all three cadences, but observation began only after the mutation had converged and the evaluator had returned a provisionally consistent classification. These data therefore test post-transition steady state, not the transition itself.

The steady-state controls produced 543 evaluator polls, zero substantive alarms, and zero epistemic warnings (Table IX). This establishes quiet post-transition behavior, but by itself does not support the transition claim in **LH3**. It does not bound false-positive rates under untested production churn; with zero observed alarms, the defensible statement is $0/543$ polls and $0/20$ windows, not a claim of zero population probability.

TABLE X
TRANSITION-INCLUSIVE BENIGN CONTROLS. NON-CONFIGURATION GOVERNANCE-PLANE DRIFT COUNTS SUBSTANTIVE POLICY, AUTHORIZATION, INTENT, OR ENVIRONMENT OUTCOMES; CONFIGURATION-CONVERGENCE AND EPISTEMIC OUTCOMES ARE REPORTED SEPARATELY.

Control	Windows	Polls	Non-configuration governance-plane drift	Config. convergence	Epistemic
C1	4	129	0	0	0
C2	4	195	0	7	12
C3	3	146	0	0	7
C4	3	105	0	0	9
C5	3	102	0	0	5
C6	3	100	0	0	3

G. Transition-Inclusive Benign Controls

We therefore re-executed the same six families with observers already active before each mutation. Twenty windows followed the same near-balanced round-robin allocation (4,4,3,3,3,3); each cadence ran in an isolated process and wrote append-only NDJSON per poll. For C2, the rollback commit and artifact digest were authorized before actuation. For C3, the exception effect and live record were retired before expiry. The harness continued for 12 s after a provisionally consistent classification and retained scheduled, start, completion, lag, component class set, and diagnostic detail for every poll.

Across 777 polls, the non-configuration governance-plane drift count was zero: no observer emitted substantive policy, authorization, intent, or environment drift (Table X). Seven C2 polls correctly exposed temporary desired/observed configuration divergence during the approved rollback. Thirty-six polls across C2–C6 returned *undecidable* while an active pod lacked a complete ready-container digest set; none laundered incomplete lineage into "consistent." Every one of the 60 window–cadence trajectories ended with a provisionally consistent classification. Thus **LH3** is supported in its safety form, while the stronger claim of silence during benign transitions is refuted. This distinction is operationally desirable: approved change does not make missing evidence true, and uncertainty is not a false substantive accusation.

H. Compound Class Sets and Extended Benign Windows

The primary campaign exercises each atomic class, but simultaneous changes can become observable at different times. We therefore added a secondary extension specified in its frozen campaign protocol: five repetitions each of live S10 (policy supersession plus expired exception), S11 (artifact substitution plus environment change), and S12 (rollback plus missing live status for continuing authority), observed at the same three cadences. These 15 injections yield 45 correlated cadence observations. After the first non-consistent verdict, the harness continues polling until the complete expected provisional class set appears or the 180-s timeout expires. We call the interval from onset to that first exact event exact-set completion latency

(ESC); DDL remains time to the first honest alert. This campaign does not impose the dwell/watermark condition of Stable-VCL (Metric M2), so it measures first completion rather than permanence. Unreached DDL/ESC values are stored as right-censored rather than assigned the 180-s timeout.

All 45 observations detected drift and reached the exact expected provisional class set (Table XI). S11 and S12 were complete at the first alert in 15/15 observations each. S10 was complete at the first alert in only 5/15: exception expiry was immediately visible, whereas Kyverno's asynchronous background scan exposed the policy component later. Its median DDL was 1.54 s, but median ESC was 7.61 s (P95 14.23 s); median diagnostic gap was 5.98 s (P95 10.03 s). These P95 values use nearest rank over 15 correlated cadence observations from five injected episodes; at $n = 15$, they equal the sample maxima and are descriptive maxima, not independent tail estimates. This is the intended result of retaining a class set over time: an early correct subset must not be mistaken for a complete diagnosis. In S12, rollback breaks the historically authorized intent lineage while missing continuing- authorization status makes authorization undecidable; the reported substantive and epistemic classes are therefore intent plus evidence. As throughout the live campaign, these are provisional sequential-snapshot classifications, not formal-cut evaluations.

The extension then ran one separate 300-s window for each benign control, 1,800 s in total. The three cadences produced 4,680 evaluator polls, zero substantive alarms, and zero epistemic warnings. The experimental units are the six windows, not the correlated polls. This is six five-minute windows, not one continuous 30-minute soak; the defensible statement remains the observed $0/6$ windows and $0/4{,}680$ polls. The longer campaign improves coverage of sustained benign operation but still does not estimate a production false-alarm probability or exercise high-volume concurrent churn.

I. Fault Contracts Through a Real Local Transport

The healthy-stream campaign is complemented by 19 deterministic evaluator tests and a controlled two-hop TCP path on `127.0.0.1`: sender to fault relay to evidence gateway. Authenticated envelopes carry stream, stable subject, sequence, capture time, payload hash, and receiver-stamped delivery time. Nine profiles exercise baseline, delay+jitter, application-record loss, duplicate, reorder, stale replay, TCP outage+identical retry, delay+loss, and invalid-lineage dependency masking. The append-only trace contains 636 events and 25 vector evaluations.

All nine profiles passed: 11 fault-state safety checks produced zero safety violations and zero component-local masking failures (Table XII). One TCP failure retried the identical payload hash; two application-record drops were counted separately. Duplicates were idempotent, reordering never regressed accepted sequence, and stale replay did not renew freshness. All recovery profiles returned to consistency; their maximum logical recovery was 0.2 s (0.1 s for outage). Wall-clock values are descriptive—only jitter has five samples (median 53.22 ms). This validates gateway safety under controlled localhost envelopes and a logical clock, not production network reliability, throughput, or cloud/Kubernetes latency.

J. Batched-Join Scaling Microbenchmark

The live evaluator is intentionally transparent but launches per-resource commands; extrapolating its one-deployment latency would confound process and API overhead with join cost. We therefore separately benchmark the dependency-free in-memory batch core over synthetic, fully decoded `UnitRef`/`EvidenceBundle` inputs. One prepared sweep builds UID-scoped approval and pod indices, then checks every container digest. The campaign uses $n \in \{1, 10, 25, 50, 100, 250, 500, 1000\}$ and 40 deterministic timing repetitions per size. Each repetition measures full-sweep, all-unit fan-out, and 10%-subset fan-out paths; those three within-repetition timings are correlated, yielding 960 path-specific raw samples rather than 960 independent experimental units.

Every sampled vector matched its seeded oracle. At $n = 1000$, the full-sweep median was 6.067 ms (P95 6.881, P99 6.927), or 164,828 units/s; an all-unit fan-out had P95 6.197 ms, while a 100-unit indexed fan-out had P95 0.618 ms. The call plan is six batched fetches per sweep versus $6n$ for a naive per-unit adapter. These numbers are consistent with the $O(n)$ call-count analysis and validate exact indexed execution over the tested values of n; they are not a Kubernetes throughput, API-quota, or production-capacity claim. The raw samples and the excluded-cost statement ship with the artifact.

K. Live Multi-Deployment Object Path

To replace purely synthetic fan-out with a bounded live object path, a second campaign created digest-locked two-container Deployments in an exclusive Kind namespace. Each timed sweep issues one `kubectl` command containing one Deployment LIST and one Pod LIST, decodes the returned JSON, preserves real namespace/UID identity and all ready-container digests, constructs scoped local approval/policy/inventory evidence, and invokes the same prepared batch core. Twenty sweeps per completed size span settled baseline, a real scale-up of a deterministic 20% cohort, and rollout restart of that cohort; mutations are phase-separated and settle before timing. The frozen campaign protocol's declared stop rules bound readiness, duration, CPU, memory, API errors, and semantic errors.

Targets $n = 10$ and $n = 50$ completed 20 sweeps each (Table XIV). The larger completed case spans 50–60 live Pods and 100–120 ready containers. Across 40 timed sweeps, 1,200 repeated per-unit baseline decisions were decidable and exact; a policy-like indexed fan-out over 20% of the fleet added 240/240 nested exact decisions. These are projected provisional decisions under the campaign's phase-separated snapshot, with zero measured API-command error and no false substantive or epistemic result. At $n = 50$, fetch dominated: P50/P95 total time was 108.61/187.75 ms, of which the semantic core used 0.332/0.415 ms. The reported 461 units/s reflects

TABLE XI
SECONDARY COMPOUND-SCENARIO EXTENSION ACROSS FIVE INJECTIONS AND THREE CADENCES PER SCENARIO. INJECTIONS AND CORRELATED CADENCE OBSERVATIONS ARE SHOWN SEPARATELY. DR COUNTS FIRST-ALERT DETECTION AND EXACT COUNTS FIRST EXACT CLASS-SET COMPLETION; "FIRST COMPLETE" IS THE FRACTION WHOSE FIRST ALERT ALREADY CONTAINED THAT SET. DDL, ESC, AND DG ARE SECONDS.

ID	Expected set	Inj.	Obs.	DR	Exact	First complete	Med. DDL	Med. ESC	Med. DG
S10	{policy, auth.}	5	15	100.0%	100.0%	33.3%	1.54	7.61	5.98
S11	{auth., environment}	5	15	100.0%	100.0%	100.0%	1.29	1.29	0.00
S12	{intent, evidence}	5	15	100.0%	100.0%	100.0%	0.69	0.69	0.00

TABLE XII
CONTROLLED LOCALHOST EVIDENCE-TRANSPORT FAULT CAMPAIGN. SAFETY MEANS NO COMPONENT IS REPORTED CONSISTENT FROM INVALID/UNAVAILABLE REQUIRED EVIDENCE; MASKING REQUIRES UNRELATED COMPONENTS TO REMAIN DECIDABLE.

Profile	Fault layer	Target	Checks	Viol.	Mask fail	Outcome
baseline	none	none	0	0	0	PASS
delay_jitter	delay+jitter	policy	5	0	0	PASS
record_drop	record loss	policy	1	0	0	PASS
duplicate	record duplicate	authorization	0	0	0	PASS
reorder	record reorder	policy	0	0	0	PASS
stale_replay	record replay	inventory	1	0	0	PASS
outage	TCP outage+retry	authorization	1	0	0	PASS
delay_drop	delay+record loss	policy	2	0	0	PASS
dependency_masking	gateway validation	lineage	1	0	0	PASS

amortization of fixed `kubectl` startup, not superlinear capacity.

The requested $n = 100$ target did not settle within the declared 240-s readiness rule, so no timed sample at that size was admitted and the campaign stopped. The archived node capacity is 110 Pods, so capacity is a plausible explanation, but the campaign captured no scheduler diagnosis at the timeout and therefore assigns no cause. Cleanup then removed only the experimental namespace and verified its absence. This campaign exercises real Kubernetes Deployments, Pods, UIDs, digests, object volume, and phase-separated churn; Flux/Kyverno and production approval, policy, and inventory services are outside this adapter-path measurement.

L. Bounded Cross-Stack Replication

To test whether representative evidence paths survive a change of reconciler and policy engine, we executed a frozen campaign on a separate Kind v0.32.0/Kubernetes v1.36.1 cluster with Argo CD v3.4.2 [3] and Gatekeeper v3.22.2 [27]. Tagged upstream manifests are retained and verified by SHA-256. Five repetitions each exercised S1 through native Argo CD desired/live status, S3 through a fresh Gatekeeper dry-run background-audit violation whose controlled Rego message emits the evaluated Deployment UID, and S4 through the existing running-digest/approved-digest adapter. The policy adapter accepts a polar result only when that engine-emitted UID matches the live Deployment UID; absence or mismatch is undecidable. Every repetition began from an exact empty projected baseline after an explicit Argo sync and leaf-by-leaf desired/live comparison. A 0.5-s poll cadence and declared readiness/resource stop rules bounded the campaign.

All 15 observations reached the exact expected singleton class set over the declared evaluated components (Table XV). The timing distinction is material. Operational-onset-to-first-honest latency ends at the earliest completed evaluation that is either substantively inconsistent or epistemically *undecidable*; it is therefore the earlier available event on two separately retained clocks, not necessarily a class-detection time. It is not DDL as defined in Metric M2, because DDL begins at the onset of the class or evidence failure being reported. First epistemic alert is the first completed evaluation with an undecidable evaluated component. First substantive alert is the first completed evaluation with an inconsistent evaluated component. ESC ends only when every declared evaluated component is decidable and the projected class set is exactly the expected singleton; it is first completion, not watermark-qualified Stable-VCL.

For S1, median operational-onset-to-first-honest, first-substantive, and ESC latencies were 0.207, 0.646, and 1.677 s; all 5 observations produced an epistemic alert, with median 0.207 s. For S3, the corresponding medians were 0.157, 1.659, and 1.659 s; again, all 5 first-honest verdicts were epistemic, with median 0.157 s. For S4 they were 0.215, 0.215, and 0.650 s; 3/5 observations also produced an epistemic alert, whose reached-sample median was 0.216 s. These latencies are reconstructed directly from the explicit onset marker and each poll's evaluation-completion timestamp in the append-only raw trace; the analyzer requires equality with the observation rows and generated summary.

All 15 baseline restorations reached `Succeeded`; the leaf-by-leaf pinned desired/live comparison found zero post-restoration differences. Instrumented reads recorded zero API errors, every final observation had zero undecidable evalu-

TABLE XIII
IN-MEMORY BATCHED-JOIN MICROBENCHMARK. CALLS ARE ARCHITECTURAL COUNTS, NOT OBSERVED API REQUESTS; KUBERNETES, NETWORK, PARSING, CONTROLLERS, AND PERSISTENCE ARE EXCLUDED.

n	Reps	P50 ms	P95 ms	P99 ms	Units/s	Batch calls*	Per-unit calls*
1	40	0.007	0.007	0.008	150 054	6	6
10	40	0.057	0.062	0.083	174 226	6	60
25	40	0.142	0.152	0.169	175 537	6	150
50	40	0.284	0.322	0.605	175 925	6	300
100	40	0.567	0.587	0.659	176 444	6	600
250	40	1.420	1.507	1.578	176 067	6	1500
500	40	2.953	3.325	3.405	169 335	6	3000
1000	40	6.067	6.881	6.927	164 828	6	6000

TABLE XIV
LIVE KIND MULTI-DEPLOYMENT ADAPTER CAMPAIGN. TIMES ARE MILLISECONDS; PODS AND CONTAINERS SHOW THE SETTLED RANGE ACROSS BASELINE AND CHURN PHASES. FETCH INCLUDES KUBECTL PROCESS STARTUP AND TWO MODELED LIST OPERATIONS.

n	Sweeps	Pods	Containers	Fetch P50	Fetch P95	Parse P50	Parse P95	Core P50	Core P95	Total P50	Total P95	Units/s
10	20	10–12	20–24	52.20	59.17	0.65	0.79	0.101	0.172	53.01	59.95	189
50	20	50–60	100–120	105.45	183.65	2.93	4.65	0.332	0.415	108.61	187.75	461

ated components, and the sustained resource stop rule did not trigger. Cleanup evidence records the exact pre-check, cluster-scoped deletion, and post-check and verifies that only `govdrift-cross` was removed.

The inferential boundary is strict. S1 and S3 are native cross-stack component-path replications for configuration and policy. S4 reuses the shared T3 digest adapter and is not independent Argo CD/Gatekeeper authorization validation; intent and environment were not evaluated. For S3, Gatekeeper's violation lacked a structural resource-UID field, so the controlled Rego rule emitted the evaluated Deployment UID in its message and the adapter accepted a polar result only when it matched the live UID. That join covers the one Deployment lifetime exercised here; it does not establish continuation-safe identity across deletion and recreation. These 15 controlled observations support descriptive concordance on three projected singleton slices, not equivalence, non-inferiority, natural prevalence, reliability, or production latency. The frozen raw polls, injection/onset markers, installation trace, upstream manifests, source-state hashes, resource samples, analyzer outputs, and cleanup proof are retained under `lab/results_cross_stack/`.

M. Evidence Boundary and Refutation Conditions

The repeated execution strengthens the earlier one-pass realizability check but remains controlled and single-cluster. It uses injected scenarios, file-backed approval and environment streams, a single deployment, no concurrent workload churn during positive injections, and only six separate five-minute benign windows. The evaluator degrades unavailable PolicyReport, approval, lineage, and inventory evidence to *undecidable*. The separate localhost relay exercises real TCP delivery but deterministic synthetic records; neither it nor the cluster campaign estimates production transport reliability, API saturation, or stale-cache races. The cluster campaign estimates a distribution only for this pinned primary laboratory protocol. The bounded Argo CD/Gatekeeper campaign covers native S1 configuration and S3 policy paths; S4 remains a shared digest adapter rather than an independent authorization replication. General authorization, intent, environment, production-backed approval/inventory services, and full cross-stack equivalence remain external-validity work rather than claims smuggled into this execution.

The expected tier/class shape appeared whenever an injection was observed, including every compound extension, supporting **LH1**. The two S1 misses support the cadence-bounded qualification in **LH2** and refute any stronger claim of cadence-independent detection. The transition-inclusive controls support **LH3**'s safety form: no policy, authorization, intent, or environment verdict; honest temporary configuration/evidence outcomes; and recovery of every trajectory. The minimal evidence bundle is constructed synchronously after verdict, so this laboratory does not test a separate, platform-bound evidence retrieval path. Incorrect exact-set output for an observed higher-tier scenario, alarms under these benign controls, or a repeatable class requiring less evidence than Proposition 4 states would refute the corresponding claims.

VIII. METRICS: PROPOSAL AND CRITICAL EVALUATION

We evaluate six candidate metrics against grounding in the model, operationalizability, and incentive safety, using the explicit rule that an unjustified proposal is rejected rather than retained for convenience.

TABLE XV
BOUNDED ARGO CD/GATEKEEPER CROSS-STACK REPLICATION. EXACT IS THE PROJECTED CLASS-SET COUNT; OPERATIONAL-ONSET-TO-FIRST-HONEST, FIRST SUBSTANTIVE, AND ESC LATENCIES ARE MEASURED TO EVALUATION COMPLETION, IN SECONDS.

Slice	Expected	Evidence surface	n	Exact	Median honest	Median substantive	Median ESC	Min honest	Max honest
					seconds				
S1	configuration	argocd-native	5	5	0.207	0.646	1.677	0.183	0.880
S3	policy	gatekeeper-native	5	5	0.157	1.659	1.659	0.142	0.332
S4	authorization	shared-artifact-adapter	5	5	0.215	0.215	0.650	0.187	0.240

Metric M1 (Governance Drift Distance, GDD)*:* Proposed as*: a scalar distance between approved and operational state.* Verdict: rejected as primitive; adopted as vector. *By Section IV-C, the primitive report is* $\big(\mathsf{GD}_{\mathrm{sub}}(u,\theta),\mathsf{O}_\mu(u,\theta;T)\big)$*; a scalar GDD is legitimate only as a published-weight composite over the decidable substantive components, with coverage reported separately and the weights understood as policy. It never substitutes for the component-resolved report. (This mirrors the model's rejection of a scalar distance and is the suite's structural decision.)*

Metric M2 (First-alert, Exact-set Completion, and Stable-VCL)*: Per class and tier: detection time minus drift onset—in events for closed-world evaluation (measured in Section VI: 0 events at the correct tier; 22.6 for naive comparison on S1), in wall-clock for the laboratory.* Verdict: adopted. *Let* t_{inject} *be command initiation and select the episode- and class-specific* $t_{\mathrm{onset}} \in \{t^{\mathrm{sub}}_{k,j}, t^{\mathrm{evd}}_{k,j}\}$ *from Definition 9: the former begins substantive divergence and the latter begins evidence failure. Let* t_{first} *be the first honest polar alert or undecidable evidence verdict. Let* $\mathscr{P}$ *be the protocol's declared completion projection—the class-set projection in our experiments, or the identity for a full-vector target—and let* $\mathscr{T}^*$ *be its declared target. Then* t_{complete} *is the first report satisfying* $\mathscr{P}(\mathscr{V}) = \mathscr{T}^*$*, and* t_{evidence} *is completion of the atomic bundle supporting that report. For a declared episode cut* θ^**, Stable-VCL measures finality of a target-valued bundle explicitly labeled historical as-of-*θ^**, not currentness at* t_{stable}*. For a candidate publication time c, let* $\mathscr{R}_{u,\theta^*}[c,T]$ *contain the bundle published at c and every subsequent evaluator revision of that unit and cut induced by a processed arrival admissible to* θ^* *through T, whether or not the revision is emitted as a report. Write* $\mathrm{Published}^{H}_{\mathscr{T}^*}(c)$ *when the bundle published at c is explicitly historical and has that unit, cut, and target projection. Write* $\mathrm{Sealed}_{u,\theta^*}(T)$ *only when every relevant watermark event and every received record behind those watermarks that is admissible to* θ^* *has been processed and materialized in the evaluator state by T, and the resulting historical bundle is sealed. We then define*

$$\mathrm{TargetHeld}(c,T) \iff \forall \mathscr{V} \in \mathscr{R}_{u,\theta^*}[c,T],\ \mathscr{P}(\mathscr{V}) = \mathscr{T}^*$$

$$t_{\mathrm{stable}} = \inf\left\{ T \;\middle|\; \begin{array}{l} \exists c \in [t_{\mathrm{complete}}, T] : \mathrm{Published}^{H}_{\mathscr{T}^*}(c), \\ \mathrm{TargetHeld}(c,T), \quad \mathrm{Sealed}_{u,\theta^*}(T), \\ \qquad \bigwedge_j \mathrm{AdmCut}_j(u,\theta^*;T) \end{array} \right\}.$$

An admissible arrival that changes the projection away from $\mathscr{T}^*$ *invalidates the current Stable-VCL completion candidate; the next matching bundle starts a new candidate interval. If no target-valued candidate reaches the sealed condition, Stable-VCL is right-censored. This restart rule does not rewrite the separately reported first-completion ESC. Because the target is preserved through every same-cut revision from c to sealing and every required source watermark has passed* θ^**, no later record admissible to that cut can revise the sealed target vector; later cuts are new evaluations, not revisions of this one. We report actuation latency* $t_{\mathrm{onset}} - t_{\mathrm{inject}}$*, DDL* $t_{\mathrm{first}} - t_{\mathrm{onset}}$*, protocol exact-set completion latency* $\mathrm{ESC} = t_{\mathrm{complete}} - t_{\mathrm{onset}}$*, diagnostic gap* $\mathrm{DG} = t_{\mathrm{complete}} - t_{\mathrm{first}}$*, end-to-end latency* $t_{\mathrm{first}} - t_{\mathrm{inject}}$*, and time-to-evidence (TTE)* $t_{\mathrm{evidence}} - t_{\mathrm{onset}}$ *separately. Stable vector-completion latency is* $\text{Stable-VCL} = t_{\mathrm{stable}} - t_{\mathrm{onset}}$*; its stable diagnostic gap is* $t_{\mathrm{stable}} - t_{\mathrm{first}}$*. In a controlled experiment, first completeness is equality to the class-set target specified in the frozen campaign protocol. The laboratory reports this provisional protocol-conditioned quantity as ESC, not VCL or Stable-VCL. Finality for the declared episode requires the cut and watermarks above; without them Stable-VCL is right-censored and the historical-finality claim is undecidable, never inferred from a quiet period. A separate verdict labeled current must satisfy* $\bigwedge_j \mathrm{CurCut}_j(u,\theta;T)$ *at its own observation time. Onset must declare its episode, class, and kind (e.g., the expiry instant is the substantive onset for exception drift, whereas loss of required status is an evidence onset); the definition ships with the metric. The original one-pass table measured injection-to-verdict and is retained only as a legacy observation in the artifact, not relabeled as DDL.*

Metric M3 (Unapproved Change Ratio, UCR)*: Fraction of observed change events (configuration and environment) not covered by a valid authorization basis at the time of effect.* Verdict: adopted, with an evidence caveat*: UCR is computable only where approvals are recorded durably; where they are not, UCR's denominator is measurable but its numerator degrades to undecidable, and the metric must report the undecidable mass separately through the relevant entry of* $\mathsf{O}_\mu(u,\theta;T)$ *rather than folding it into either pole. With N observed changes, U known unapproved changes, and Q undecidable changes, report the identification interval* $\mathrm{UCR}_{\min} = U/N$ *and* $\mathrm{UCR}_{\max} = (U+Q)/N$ *rather than a fabricated point estimate. If* $N = 0$*, both endpoints are undefined: report that no change exposure was observed, not a zero UCR.*

Metric M4 (Policy-Version Divergence, PVD)*: For each*

running deployment: the set (and count) of policy controls, in the currently governing version, that the deployment violates while remaining covered only by an older pinned basis—estate-wise, the distribution of "policy age" (versions or time between pinned and current). Verdict: adopted as a set/age pair; rejected as a bare count*, since control violations are policy-weighted and a count of one critical control must not compare equal to one cosmetic control.*

Metric M5 (Authorization Drift Ratio, ADR)*: Let $E_A(t)$ be the set of active authorization-dependency edges $e = (u, \eta, \rho)$ from an operational effect η to one authorization obligation ρ. For each edge, let $\mathscr{M}_e$ be the family of inclusion-minimal instrument sets that can satisfy that obligation. Alternatives therefore do not become extra denominator items. Under the three-valued semantics of Section IV-A, including subject and scope coverage, define*

$$v_e(t) = \bigvee_{M \in \mathscr{M}_e} \bigwedge_{a \in M} \text{Applicable}(a, \tau_e, t).$$

Thus one true minimal satisfying set covers the edge; the edge is false only when every alternative is false, and is $\perp$ when no alternative is known true and at least one remains undecidable. If the edge or its minimal-set family cannot itself be established, set $v_e = \perp$. Let $N = |E_A|$, F be the number of false edges, Q the number of $\perp$ edges, and $D = N - Q$. Report

$$\text{ADR}_{\text{dec}} = F/D, \qquad \text{ADR-coverage} = D/N,$$

with the identification interval $[F/N, (F+Q)/N]$ over all active edges; a zero denominator is undefined, not zero. Verdict: adopted. *The denominator is dependency edges, never raw approval rows or the number of alternative minimal satisfying sets, so duplicate or redundant instruments cannot improve ADR mechanically.*

Metric M6 (Governance Conformance Coverage, GCC)*: Let $X(T)$ be the running deployments at observation time T, let u_x be the stable unit reference for x, and let $\theta_x(T)$ be its declared current logical cut. A deployment is operationally evaluable exactly when a unique admitted basis exists and every component satisfies both its evidence contract and its current-cut contract:*

$$\begin{aligned}\text{Eval}_{\text{cur}}(x,T) \iff & B_x(\theta_x(T)) \neq \perp \\ & \wedge \bigwedge_{j \in J} o_{\text{current},j}(u_x, \theta_x(T); T) = 1 \\ & \wedge \bigwedge_{j \in J} \text{CurCut}_j(u_x, \theta_x(T); T),\end{aligned}$$

$$\text{GCC}_{\text{cur}}(T) = \frac{\sum_{x \in X(T)} \mathbf{1}[\text{Eval}_{\text{cur}}(x,T)]}{|X(T)|}.$$

If $|X(T)| = 0$, GCC_{cur} is undefined, not zero. This is the operational GCC headline; an admissible but stale historical cut cannot enter its numerator.

Historical audit coverage is a separate estimand. For a declared target set $\Theta = \{(x, \theta_x)\}$ and audit observation time T, replace CurCut *above by* AdmCut *and the current mask by the historical mask:*

$$\begin{aligned}\text{Eval}_{\text{hist}}(x, \theta_x; T) \iff & B_x(\theta_x) \neq \perp \\ & \wedge \bigwedge_{j \in J} o_{\text{historical},j}(u_x, \theta_x; T) = 1 \\ & \wedge \bigwedge_{j \in J} \text{AdmCut}_j(u_x, \theta_x; T),\end{aligned}$$

$$\text{GCC}_{\text{hist}}(\Theta; T) = \frac{\sum_{(x,\theta_x) \in \Theta} \mathbf{1}[\text{Eval}_{\text{hist}}(x, \theta_x; T)]}{|\Theta|}.$$

It is undefined for $|\Theta| = 0$ and must be labeled with both target cuts and audit observation time; it is never presented as current coverage. Verdict: adopted, and placed first in reporting order*: coverage bounds the meaning of every other metric (a superb ADR over 20% current GCC is a statement about 20% of the running estate). Low GCC means that at least one required evidence contract failed—basis selection, identity linkage, source presence, completeness, integrity, authentication, freshness, health, or temporal-cut closure. Missing approved-state recording is one important cause, not the definition of the failure.*

Reporting discipline: current GCC first; any historical GCC next and labeled with its target cuts and audit time; then the class-resolved vector metrics (DDL per tier, ESC for protocol targets, Stable-VCL only with its cut-and-watermark qualification, UCR with undecidable mass, PVD as set/age, ADR with edge coverage and bounds); composites last, weights published. The gaming surfaces are the usual ones—weakening policy lowers PVD; widening approvals lowers UCR—and, as in any governance measurement, the counters are meaningful only jointly with the policy and approval baselines they are computed against.

A. What the Present Data Can and Cannot Instantiate

The live campaign identifies DDL and exercises component-mask behavior. It instantiates neither GCC estimand defined above: its units are repeated observations rather than a running-deployment estate, and its synchronous readers do not claim the per-component current-cut/watermark contract. Each of the 60 S9 cadence observations intentionally fails the current-authorization live-status contract while retaining immutable execution proof. The resulting 480/540 (88.9%) is protocol-level class-set evaluability in the balanced positive set, not GCC. Separately, the post-stabilization benign-control campaign recorded 543/543 evaluable polls. Both are contract exercises, not estate coverage estimates. UCR, PVD, and ADR are not reported from these injections: their denominators are deployment populations or naturally occurring change and authorization-dependency edges, whereas this case-control design fixes scenario counts by construction. Relabeling those fractions as operational rates would manufacture prevalence.

For an estate, aggregate only after reporting current GCC: compute each metric over evaluable deployments, retain unevaluable mass separately, and stratify by service criticality, environment, and evidence tier. A fleet-wide scalar may then be

a published-weight summary, but it must not erase class counts or let a high-coverage low-risk stratum conceal an unevaluable critical stratum.

IX. Related Work

A. The Missing Join

The closest systems do observe many of the ingredients of governance drift, but they organize them around plane-local questions: desired versus observed configuration, state versus current policy, artifact versus admission rule, or environment versus benchmark. Table XVI makes the gap explicit. The table compares each line of work by its *native decision object*, not by every integration a commercial product might expose. A triangle means that the line carries a useful slice but does not natively join that slice to the time-indexed admitted basis.

This fragmentation explains why the aggregate relation remained easy to miss: a deployment can be green in each available local view while the join among present state, current context, and the recorded admitted basis is broken. The contribution is not the claim that no existing mechanism can be composed to cover several columns. It is the explicit temporal three-way relation, the class-resolved verdicts it yields, and the evidence coverage required to decide it.

B. Terminology and Review Boundary

The phrase *governance drift* predates this work, and we make no claim to have coined it. AWS Control Tower officially uses the phrase (also "organizational drift") for changes to managed organizational units, accounts, landing-zone artifacts, controls, policies, and inherited baselines [28]. That product taxonomy is an important precedent. Its documented decision object is nevertheless Control Tower-managed organizational and control state; AWS explicitly excludes resource drift inside a baseline. It does not define a deployment-specific join between present workload state, evolving governance context, and a uniquely activated immutable approval record.

Two 2026 papers further sharpen the boundary. Marella's SSRN preprint proposes centralized policy abstraction, cross-cloud policy mapping, and continuous drift detection across AWS, Azure, and GCP [29]. Its object is provider-independent policy consistency; that layer can supply T1 and T4 evidence, but does not by itself establish whether a running deployment remains covered by the authorization, intent, and environmental assumptions recorded at admission. Do et al. use the exact phrase *AI governance drift* for post-deployment misalignment between governance mechanisms and the behavior of deployed AI systems, and evaluate an organizational model relating audit, monitoring, assurance, visibility, drift, and resilience [31]. Their unit of analysis is AI governance capability and behavior rather than a cloud deployment's immutable admitted basis; for that reason, the work is discussed here but is not forced into the mechanism columns of Table XVI.

Other 2026 uses confirm that the compound is polysemous rather than available for a naming claim. Höpfner and Schacht study *socio-technical governance drift* as longitudinal erosion of rule compliance in human–AI teams [32]. Solozobov's preprint and accompanying toolkit monitor degradation of the statistical evidence supporting deployed risk models under delayed ground truth [33]. The former concerns organizational oversight behavior; the latter concerns model performance evidence. Neither makes cloud workload admission history its decision object, but both are relevant neighbors and are included rather than dismissed as mere keyword collisions.

We tested the remaining novelty statement with a bounded, one-reviewer *scoping review* updated through August 8, 2026—not a systematic review or a claim of corpus completeness. The protocol records the exact queries, sources, cutoff, inclusion criteria, and limitations; its item-level ledger records the included and excluded candidates and a reason for every decision (`literature/scoping_review_protocol.md` and `literature/scoping_review_records.csv`). Discovery used exact-term and adjacent-construct queries, while substantive claims were checked against official documentation, standards, DOI/publisher records, and primary proceedings. No corpus-size estimate, dual screening, or risk-of-bias appraisal is implied. The bounded conclusion is correspondingly narrow: among the screened sources, none jointly specifies the full deployment-level, history-anchored, class-resolved relation defined here.

C. Configuration Drift and Desired-State Systems

Configuration drift detection is mature practice: reconciler status in GitOps [3, 4, 1, 2], cloud configuration recorders [6], IaC drift tools [7], and the configuration-management control tradition [14, 15] that institutionalizes baseline comparison. IaC research studies the quality and smells of the configuration artifacts themselves [34, 35, 36]. GitOps reconcilers and many deployed drift tools use the current desired state as their comparator, but configuration management as a field is not uniformly memoryless. NIST SP 800-128 requires an established baseline, security-impact analysis, approval and documentation of controlled changes, and monitoring of the resulting configuration [14]; it therefore supplies genuine approved-history semantics. The narrower boundary is dimensional: that history is configuration-scoped, whereas our relation joins the selected deployment admission to current policy, authorization, artifact evidence, intent, and environment as well. Proposition 1 bounds what the configuration-only vocabulary can decide, and Finding F4 measures the bound. Our claim is containment, not erasure: this literature solves the configuration component well—T0n in our architecture *is* this literature—but it does not decide all six classes from configuration evidence alone.

D. Policy-as-Code, Admission, and Continuous Compliance

Policy engines [9, 10] and admission control [8] evaluate governance predicates at a point in time; posture management [13] and hardening baselines [30] scan environments against benchmark policy; continuous-validation features in attestation-gated deployment [11] re-check the artifact-policy slice after admission. These supply, respectively, our T1 evaluation semantics, part of T4's observation, and a one-tier precedent for contin-

TABLE XVI
REPRESENTATIVE NATIVE DECISION SCOPE OF ADJACENT WORK. ✓ = NATIVE DECISION OBJECT; △ = PARTIAL INPUT OR SLICE; – = NOT NATIVE. THE COMPARISON IS NOT AN EXHAUSTIVE PRODUCT FEATURE AUDIT.

Line of work	Config.	Policy	Auth.	Intent	Evidence	Environ.	Joins admitted basis
Desired-state/GitOps reconciliation [3, 4, 1]	✓	–	–	–	–	–	–
Security-focused configuration baselines [14, 15]	✓	△	–	△	△	△	–
IaC drift/configuration recorders [6, 7]	✓	–	–	–	–	△	–
AWS Control Tower governance drift [28]	✓	✓	△	–	–	△	–
Multi-cloud policy consistency/drift [29]	✓	✓	–	–	–	✓	–
Policy-as-code/background scans [9, 10]	–	✓	–	–	–	–	–
Posture and hardening scans [13, 30]	△	✓	–	–	–	✓	–
Supply-chain attestations [19, 23]	–	△	△	△	✓	–	–
Continuous artifact validation [11]	–	✓	✓	–	△	–	–
Governance drift (this work)	✓	✓	✓	✓	✓	✓	✓

uous re-evaluation. Commercial continuous-compliance and CSPM products may integrate several of these signals, but their native decision object remains current state versus current rule or benchmark; this is not an exhaustive feature audit. What the documented mechanisms do not provide is the *approved-state reference*: they compare state against current policy, not against the basis under which the state was admitted, so they cannot distinguish "was never compliant" from "was admitted legitimately and the world moved"—the distinction carrying the remediation semantics of Section IV-C, and the defining relation of governance drift. Runtime compliance-monitoring research systematically studies rule evaluation over process-event streams and fine-grained violation feedback [37]. That literature supplies monitoring languages and execution semantics; its native reference is a compliance rule or process model, not a deployment-specific immutable admission event. The distinction is complementary rather than competitive: such monitors can implement T1, while $B_x(t)$ determines why the same current violation is classified as post-admission drift rather than absence of an admitted basis. Within the screened set, the closest new result on the authorization timeline is Peng and Wu's policy-state serializability: it prevents an authorization or delayed human approval from becoming stale while mutable policy state changes before the governed effect commits [38]. Its protected interval ends at effect commit; ours begins from the admitted deployment and asks whether the running object remains covered as multiple governance planes evolve afterward. The guarantees are complementary: serializable admission can create a sound basis, and the retained basis can support post-admission drift detection.

E. Supply-Chain Integrity and Authorization Evidence

Attestation frameworks bind artifacts to build and review facts [19, 20, 23]; they are T3's substrate and make S4/S8-class drift decidable. Their orientation is admission-time verification; governance drift adds the post-admission temporal dimension (validity windows, revocation, supersession) and the join against G_{app}. Howell and Kotz's end-to-end authorization carries the authority justifying a request across administrative, network, abstraction, and protocol boundaries [39]; governance drift preserves an analogous justification across time, but asks whether a previously admitted, still running deployment remains covered after its context changes. Provenance work provides equally direct foundations for the evidence join. Muniswamy-Reddy et al. integrate provenance across application, workflow, storage, and network layers [40]; TAP explicitly represents time, distributed state, and state changes in provenance [41]. Those systems solve cross-layer ancestry and temporal explanation, not the normative, class-resolved conformance test against an activated approval record. Records-side, durable approval evidence and transparency services [42, 43, 24] determine whether that test is decidable at all—our observability mask and GCC metric make the dependence explicit rather than assumed.

F. Access Control, Runtime Verification, and Drift Theory

Temporal validity of authorization has deep roots in temporal authorization and role models [44, 45], while usage control adds ongoing decisions, mutable attributes, obligations, and conditions [46, 47]. These works justify the one-shot/continuing distinction rather than leaving validity as a Boolean. Classical access-control theory [17, 48] and zero-trust's continuous-evaluation stance [49] provide the broader foundation; governance drift applies it to deployment admission history. Runtime verification [50, 51] monitors executions against temporal specifications; our evaluator is kin—governance consistency is a safety-style property over joined state and context streams—but the specification here is not a given formula: it is reconstructed per deployment from the approved snapshot, which is why recording G_{app} is the architecture's prerequisite rather than an optimization. Concept drift [21] shares only the word: it names distribution change in data streams, not divergence from an approval basis; we flag it to prevent terminological import of its detectors, which answer a statistical question ours does not ask. The exact-phrase precedents and review boundary are stated above so that the contribution rests neither on naming priority nor on a universal absence claim. It rests on the specified decision object and the analytic consequences of retaining the admitted basis.

G. Novelty, Answered Directly

Is this just configuration drift? No—with the argument's weight placed where it belongs. The load-bearing evidence is analytic: constructed, tool-realizable counterexamples in which configuration equality and governance inconsistency coexist (Figure 3), and the dependency result that

no configuration-pair detector decides the property (Proposition 1). The executed staircase (Table II) supports these as an implementation-validation and noise study, not as independent field evidence. And we concede the mechanical point a skeptic will press: every governance-tier detector is itself a comparison—against a recorded basis, a validity clock, or a coverage set. The claim is not a new mechanism but a different *relation*: three-place (state, current context, admitted basis) where the desired/observed configuration signal is two-place and memoryless, with the consequences Section X enumerates. *Is it policy compliance scanning?* Scanning evaluates current state against current policy; governance drift is three-way—state, current context, *and approved basis*—and the third term changes both the verdicts (legitimately-admitted-since-superseded $\neq$ never-compliant) and the remediations. *Is it continuous compliance scanning or continuous validation?* Per tier, largely yes—and we credit it: Kyverno-style background scanning continuously re-evaluates resources against current policy (T1); configuration recorders and posture management watch unmanaged resources (much of T4); continuous validation re-checks artifact policy (part of T3) [10, 6, 13, 11]. Several of our scenarios trigger *some* existing detector. Within the bounded scoping set, we found no documented mechanism providing the complete join against the admitted basis with mode-aware validity and class-resolved verdicts—the increment that turns "a scanner fired" into "this running state is no longer covered by what admitted it, for this reason, with this remediation family." That increment is architectural and, we argue, decision-relevant; quantifying its operational value on real estates remains future field work. The present laboratory tests only the three hypotheses stated in Section VII.

X. Discussion and Limitations

A. Legitimacy, Blame, and the Policy-Drift Question

Measuring divergence is not assigning fault. Policy drift in particular is usually the *organization's own* motion; flagging a deployment as governance-inconsistent under π_8 says migration or re-approval is due, not that anyone erred. We consider this a feature of the vector semantics: because classes are separated, an estate can adopt distinct dispositions per class (auto-reconcile configuration drift; ticket policy drift with a migration SLA; page on authorization drift; block on intent drift)—consistent with observational DORA evidence associating delivery practices with delivery and stability outcomes [52, 53]—turning the taxonomy into an operating policy rather than an alarm firehose. What the observational association does not establish is that the response families proposed here cause either faster delivery or greater stability. What the model deliberately does not decide—and no measurement can—is the disposition itself. Table I makes the distinction systematic: class informs a primary risk channel and response family, while blast radius, duration, reversibility, consequence, and evidence coverage determine actual severity. The vector must not be collapsed into a universal danger ranking.

B. Evidence Failure and Business Availability

An *undecidable* verdict is epistemic, not an automatic shutdown command. The safe default is fail-closed for the *governance claim*—the system must not report conformance without evidence—while availability policy is chosen separately. A new admission may be blocked when T1–T3 evidence is missing; an already-running critical service may remain available under a bounded grace period, elevated alert, and evidence-repair SLO. Conversely, a retroactive revocation or known uncovered digest is a substantive verdict, not evidence degradation, and may justify immediate containment. The component, service criticality, blast radius, and maximum tolerated evidence age must therefore select fail-open or fail-closed actuation in a published policy. This preserves business availability without laundering uncertainty into a false "consistent" state.

C. Limitations

The approved-state prerequisite. Everything downstream of Section V assumes an admitted basis $B_x(t)$ was recorded durably; where it was not, the basis-dependent components are undecidable and total consistency cannot be established. If no other component is known nonzero, total status is undecidable; if configuration or another independently established component is known nonzero, total status is inconsistent because known inconsistency precedes unknown evidence. Configuration remains comparable when its desired/observed evidence is available, but a clean configuration component cannot support a total governance-conformance claim; the honest headline is component coverage and GCC, not a total drift rate. This is a real adoption constraint, not a footnote: estates without durable approval evidence cannot measure their basis-dependent governance relations, and discovering that is arguably the measurement's first value. *Bounded live empirics.* The laboratory executed 20 repetitions of each of S1–S9 at three evaluator cadences on one Kind/Flux/Kyverno stack, plus 20 post-stabilization benign control windows. Its 538/540 detection observations, 538/538 conditional exact provisional class-set agreements, and 0.000617 unconditional Hamming loss are supplemented by 45/45 exact provisional compound-class-set completions and six separate five-minute windows totaling 4,680 steady-state polls with no substantive or epistemic alarm. A transition-inclusive rerun adds 777 per-poll observations: zero policy, authorization, intent, or environment drift; seven expected configuration-convergence polls, and 36 fail-safe evidence-warning polls; all 60 window–cadence trajectories ended with a provisionally consistent classification. The bounded Argo CD/Gatekeeper replication produced 15/15 exact projected singleton classifications over the declared evaluated components for S1, S3, and S4. Configuration and policy used native second-stack surfaces; S4 retained the shared digest adapter, and intent and environment were unevaluated. It is therefore neither complete-vector validation nor a cross-stack equivalence or independent authorization-validation result. The completed campaigns demonstrate bounded realizability and within-laboratory behavior under their reported conditions. They do not establish natural occurrence, field preva-

lence, production reliability, universal false-alarm rates, stream-loss tolerance, full-stack equivalence, or transfer across organizations. The two cadence misses are retained because they expose a real observability boundary for transient drift. *Temporal-contract implementation gap.* The formal model requires stable unit references, authenticated capture intervals, watermarks, and an admissible cut. The primary cluster evaluator reads its sources sequentially and does not implement production watermarks; namespace/UID correlation, fail-safe incomplete-rollout behavior, deterministic envelope faults, and hash-chain selection test pieces on the live path, while a separate executable cut selector tests the complete rejection semantics over synthetic stream records. Thus live outputs are provisional class-set classifications under a laboratory sequential-snapshot assumption, while temporal-cut tests validate the contract separately; neither is a claim that arbitrary distributed snapshots are coherent or that the live runs evaluated formal Consistent. *Live granularity and scale.* The scenario campaign remains per-deployment. The separate live adapter campaign covers 50 Deployments, 60 Pods, 120 containers after settled churn, and the actual Kubernetes object fetch/parse path; its attempted 100-Deployment target hit the declared readiness stop rule. Approval, policy, and inventory evidence in that campaign remain synthesized, and churn is phase-separated. The 1,000-unit microbenchmark is consistent with the $O(n)$ call-count analysis and validates exact indexed execution only over the tested values of n. API saturation, multi-cluster aggregation, production services, and positive injections under truly concurrent fleet churn remain open. *Environment scope.* d_{env} is bounded by what σ_B declares; assumptions never written down cannot drift detectably—an instance of the general dependence of governance measurement on capture discipline. *Remediation dynamics.* We model detection, not the control loop that acts on it; closing the loop (auto-re-approval workflows, drift-driven migration) is future work with its own stability questions.

D. Outlook

The machinery this work presupposes—durable approval evidence, and an account of how such evidence decays—and the drift semantics defined here compose naturally: approved-state snapshots are exactly the artifacts durable approval linkage produces, and evidence drift is evidence debt manifesting inside the drift vector. We have kept this manuscript self-contained, but the research program is one: make the relationship between what organizations approve and what their systems do continuously demonstrable—at admission, over time, and after the fact.

XI. Data and Code Availability

The version 1.6.0 release contains the manuscript source, reference implementation, closed-world data, separately implemented no-join baseline, ablation, repeated and compound live-stack observations, transition controls, fault traces, scaling samples, live-fleet records, real-process audit trace, bounded Argo CD/Gatekeeper cross-stack records, generated tables, and validators. Development history and the frozen tag `v1.6.0` are at `https://github.com/obedebessa/governance-drift`. The verified release is archived at `https://doi.org/10.5281/zenodo.21847543`; the version-independent concept DOI is `https://doi.org/10.5281/zenodo.21841458`. The release manifest binds the manuscript PDF and source package by SHA-256.

The dependency-free closed-world study is regenerated with `python3 code/detector_study.py`; the ablation with `python3 code/ablation_study.py`; and the full frozen package is checked with `python3 scripts/verify_artifact.py`. That aggregator re-executes the unit suites and validates the uncertainty, compound, transition, fault, scaling, live-fleet, trace, cross-stack, and scoping-review artifacts without recreating a cluster. With Docker, Kind, `kubectl`, Git, and the pinned images available, `lab/bootstrap.sh` creates the primary stack and `python3 lab/run_repeated_experiment.py` executes the seeded 20-by-9 protocol and steady-state controls. Platform versions, seeds, cadences, raw timestamps, censoring, expected and observed class sets, per-component metrics, Hamming loss, evidence-completion times, stop rules, cleanup checks, and source/raw hashes—and, for the trace campaign, a complete file manifest—are retained in the reportable directories `lab/results`, `lab/results_extension`, `lab/results_transition`, `lab/results_faults`, `lab/results_scaling`, `lab/results_live_fleet`, `lab/results_trace`, and `lab/results_cross_stack`. Pilot and intentionally aborted development runs are excluded from the release manifest and reported claims.

XII. Conclusion

Configuration convergence answers whether observed state matches declared desired state. It does not answer whether that state remains covered by the approval, policy, authorization, intent, and environmental assumptions that made it legitimate. This paper formalized that second question as governance drift: a deployment-level, history-anchored relation among operational reality, current governance context, and a unique activated admitted basis. The model separates five substantive components from an epistemic observability mask; distinguishes approval, activation, supersession, and authorization modes; requires stable unit identity and admissible temporal cuts; and makes ambiguity or missing evidence undecidable rather than silently consistent. Configuration equality therefore does not imply governance consistency, and a configuration-pair detector cannot decide the missing relations because their deciding facts are outside its inputs.

The empirical results test what that claim means in executable systems. In the twelve-scenario closed world, normalized configuration comparison decides one scenario, while the admitted-basis join decides all twelve at the required tiers. A separately implemented no-join comparator identifies 9/12 sets, and a cumulative ablation raises unconditional exact-vector success from 75.0% to 100% over 240 paired scenario–seed units; targeted probe coverage across B0–B4 is 3/10, 3/10, 4/10, 5/10,

and 10/10. This isolates the contributions of evidence contracts, mode-aware authorization, intent history, and activation-safe basis selection rather than rewarding alarm volume.

On the version-pinned Kind/Flux/Kyverno laboratory, 180 injections observed at three cadences produced 538/540 detections. All 538 detected outputs matched exactly (100% conditional ESA) as provisional class-set classifications under the laboratory's sequential-snapshot assumption, while unconditional Hamming loss over all 540×6 output labels was 0.000617. The two retained misses expose the cadence boundary for transient drift. The compound extension reached the exact provisional class set in 45/45 correlated observations and showed why first honest alert and complete diagnosis differ: for S10, median DDL was 1.54 s while median ESC was 7.61 s. Six separate five-minute benign windows (30 minutes cumulative) added 4,680 polls without substantive or epistemic alarms. Transition-inclusive controls retained all 777 polls: no policy, authorization, intent, or environment drift; seven configuration-convergence polls, 36 fail-safe evidence-warning polls, and all 60 window–cadence trajectories ending with a provisionally consistent classification. Separate temporal-cut tests exercise the admissibility contract; they do not make these sequential live snapshots formal Consistent evaluations.

Additional campaigns bound the implementation claim instead of inflating it. A nine-episode audit ran 27 separate observer processes and retained 261 raw polls; all 27 correlated trajectories reached the exact expected class set and then two-poll exact persistence, with zero adapter error. Cause-start-to-first-exact completion had median 4.465 s (nearest-rank P95 9.764), while cause-start-to-two-poll exact persistence had median 9.522 s (P95 19.811); the latter is not watermark-qualified Stable-VCL. A real two-hop localhost transport campaign passed all eleven fault-state safety checks. The in-memory batch core remained exact through 1,000 units, and the live object path completed a 50-Deployment target spanning 50–60 Pods and 100–120 containers; the 100-Deployment target hit its declared readiness stop and contributed no timing sample. The bounded Argo CD/Gatekeeper replication produced 15/15 exact projected singleton classifications over declared evaluated components. S1 configuration and S3 policy used native second-stack paths; S4 retained the shared digest adapter, and intent and environment were not evaluated. These results do not constitute complete-vector validation, independent S4 authorization validation, or cross-stack equivalence.

These results answer RQ0 under explicit conditions: governance drift is representable as a component-resolved vector; configuration remains comparable from desired and observed state, whereas basis-dependent components are measurable only where the admitted basis and their required evidence exist. Missing basis evidence prevents total consistency but does not erase a known inconsistency in another component. Detection reaches only the depth of the maintained evidence tiers. The results demonstrate semantic realizability, safe uncertainty handling, and bounded behavior within the reported conditions. They do not estimate natural occurrence, organizational prevalence, production reliability, or universal latency and false-alarm distributions. The practical consequence is nevertheless direct: a system can match its manifest perfectly while one or more legitimacy conditions no longer hold, and seeing that difference is an architectural choice that must be made—and reported—tier by tier.

REFERENCES

[1] OpenGitOps Project, "GitOps principles, version 1.0.0." `https://opengitops.dev/`, 2021. CNCF. Accessed August 2026.

[2] F. Beetz and S. Harrer, "GitOps: The evolution of DevOps?," *IEEE Software*, vol. 39, no. 4, pp. 70–75, 2022.

[3] Argo Project, "Argo CD: Declarative GitOps continuous delivery for Kubernetes." `https://argo-cd.readthedocs.io/`, 2026. Accessed August 2026.

[4] Flux Project, "Flux: The GitOps family of projects." `https://fluxcd.io/`, 2026. Accessed August 2026.

[5] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, omega, and Kubernetes," *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016.

[6] Amazon Web Services, "AWS Config developer guide: Detecting drift and evaluating resources." `https://docs.aws.amazon.com/config/latest/developerguide/`, 2026. Accessed August 2026.

[7] Snyk, "driftctl: Detect, track and alert on infrastructure drift." `https://docs.driftctl.com/`, 2023. Accessed August 2026.

[8] Kubernetes Project, "Admission controllers reference; validating admission policy." `https://kubernetes.io/docs/reference/access-authn-authz/`, 2026. Accessed August 2026.

[9] Open Policy Agent Project, "Open policy agent documentation." `https://www.openpolicyagent.org/docs/`, 2026. CNCF. Accessed August 2026.

[10] Kyverno Project, "Kyverno: Kubernetes native policy management." `https://kyverno.io/docs/`, 2026. Accessed August 2026.

[11] Google Cloud, "Binary authorization documentation, including continuous validation." `https://cloud.google.com/binary-authorization/docs`, 2026. Accessed August 2026.

[12] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA, USA: O'Reilly Media, 2016.

[13] Microsoft, "Cloud security posture management (CSPM) documentation, microsoft defender for cloud." https://learn.microsoft.com/en-us/azure/defender-for-cloud/concept-cloud-security-posture-management, 2026. Accessed August 2026.

[14] A. Johnson, K. Dempsey, R. Ross, S. Gupta, and D. Bailey, "Guide for security-focused configuration management of information systems," Tech. Rep. NIST Special Publication 800-128, National Institute of Standards and Technology, 2011.

[15] Joint Task Force, "Security and privacy controls for information systems and organizations," Tech. Rep. NIST Special Publication 800-53, Revision 5, National Institute of Standards and Technology, 2020.

[16] United States Congress, "Sarbanes–oxley act of 2002." Public Law 107–204, 116 Stat. 745, 2002.

[17] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[18] AXELOS, "ITIL foundation: ITIL 4 edition." TSO (The Stationery Office), 2019.

[19] OpenSSF, "SLSA specification, version 1.2." https://slsa.dev/spec/v1.2/, 2025. Accessed August 2026.

[20] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *Proceedings of the 28th USENIX Security Symposium*, pp. 1393–1410, USENIX Association, 2019.

[21] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.

[22] ControlMonkey, "Cloud governance best practices: Five ways to prevent drift." https://controlmonkey.io/blog/cloud-governance-best-practices-devops/, 2025. Representative industry usage conflating drift management with governance; accessed August 2026.

[23] Z. Newman, J. S. Meyers, and S. Torres-Arias, "Sigstore: Software signing for everybody," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 2353–2367, ACM, 2022.

[24] Amazon Web Services, "AWS CloudTrail user guide." https://docs.aws.amazon.com/awscloudtrail/latest/userguide/, 2026. Accessed August 2026.

[25] OpenTelemetry Project, "The opentelemetry specification." https://opentelemetry.io/docs/specs/otel/, 2026. CNCF. Accessed August 2026.

[26] Kubernetes SIGs, "kind: Kubernetes in Docker." https://kind.sigs.k8s.io/, 2026. Accessed August 2026.

[27] Open Policy Agent Project, "Gatekeeper: Policy controller for Kubernetes." https://open-policy-agent.github.io/gatekeeper/website/docs/, 2026. Accessed August 2026.

[28] Amazon Web Services, "Types of governance drift." AWS Control Tower User Guide, 2026. https://docs.aws.amazon.com/controltower/latest/userguide/governance-drift.html. Accessed August 8, 2026.

[29] R. Marella, "A standardized multi-cloud governance model for policy consistency and drift detection." SSRN preprint, 2026. Posted May 5, 2026.

[30] National Security Agency and Cybersecurity and Infrastructure Security Agency, "Kubernetes hardening guide, version 1.2." Cybersecurity Technical Report, 2022.

[31] T.-Y. Do, G.-N.-T. Truong, D.-Q. Bao, K. Sharma, and P.-M.-B. Nguyen, "AI governance drift: Why audit and governance controls fail after AI deployment," *EDPACS*, pp. 1–10, 2026. Published online June 15, 2026.

[32] S. Höpfner and S. Schacht, "Architecture as governance: Why LLM agents need structural compliance enforcement." OpenReview manuscript, 2026.

[33] O. Solozobov, "Label-free detection of governance evidence degradation in risk decision systems," 2026. Preprint submitted April 20, 2026.

[34] A. Rahman, R. Mahdavi-Hezaveh, and L. Williams, "A systematic mapping study of infrastructure as code research," *Information and Software Technology*, vol. 108, pp. 65–77, 2019.

[35] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pp. 164–175, IEEE, 2019.

[36] T. Sharma, M. Fragkoulis, and D. Spinellis, "Does your configuration code smell?," in *Proceedings of the 13th International Conference on Mining Software Repositories (MSR)*, pp. 189–200, ACM, 2016.

[37] L. T. Ly, F. M. Maggi, M. Montali, S. Rinderle-Ma, and W. M. P. van der Aalst, "Compliance monitoring in business processes: Functionalities, application, and tool-support," *Information Systems*, vol. 54, pp. 209–234, 2015.

[38] Y. Peng and X. Wu, "Stateful governance for concurrent agentic systems," 2026. Preprint submitted August 3, 2026.

[39] J. Howell and D. Kotz, "End-to-end authorization," in *Fourth Symposium on Operating Systems Design and Implementation (OSDI 2000)*, (San Diego, CA, USA), USENIX Association, Oct. 2000.

[40] K.-K. Muniswamy-Reddy, U. Braun, D. A. Holland, P. Macko, D. Maclean, D. Margo, M. Seltzer, and R. Smogor, "Layering in provenance systems," in *2009 USENIX Annual Technical Conference (USENIX ATC 09)*, (San Diego, CA, USA), USENIX Association, June 2009.

[41] W. Zhou, L. Ding, A. Haeberlen, Z. Ives, and B. T. Loo, "TAP: Time-aware provenance for distributed systems," in *3rd USENIX Workshop on the Theory and Practice of Provenance (TaPP 11)*, (Heraklion, Crete, Greece), USENIX Association, June 2011.

[42] H. Birkholz, A. Delignat-Lavaud, C. Fournet, Y. Deshpande, and S. Lasker, "An architecture for trustworthy and transparent digital supply chains (SCITT)." Internet-Draft draft-ietf-scitt-architecture-22, IETF, 2025. Work in Progress, October 2025.

[43] L. Moreau and P. Missier, "PROV-DM: The PROV data model." W3C Recommendation, 2013. `https://www.w3.org/TR/prov-dm/`. April 30, 2013.

[44] E. Bertino, C. Bettini, and P. Samarati, "A temporal authorization model," in *Proceedings of the 2nd ACM Conference on Computer and Communications Security*, pp. 126–135, 1994.

[45] E. Bertino, P. A. Bonatti, and E. Ferrari, "TRBAC: A temporal role-based access control model," *ACM Transactions on Information and System Security*, vol. 4, no. 3, pp. 191–233, 2001.

[46] J. Park and R. Sandhu, "The UCONABC usage control model," *ACM Transactions on Information and System Security*, vol. 7, no. 1, pp. 128–174, 2004.

[47] U. Schöpp, C. Xu, A. Ibrahim, F. Faghih, and T. Dimitrakos, "Specifying a usage control system," in *Proceedings of the 28th ACM Symposium on Access Control Models and Technologies*, 2023.

[48] D. D. Clark and D. R. Wilson, "A comparison of commercial and military computer security policies," in *Proceedings of the 1987 IEEE Symposium on Security and Privacy*, pp. 184–194, IEEE, 1987.

[49] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," Tech. Rep. NIST Special Publication 800-207, National Institute of Standards and Technology, 2020.

[50] M. Leucker and C. Schallhart, "A brief account of runtime verification," *Journal of Logic and Algebraic Programming*, vol. 78, no. 5, pp. 293–303, 2009.

[51] A. Pnueli, "The temporal logic of programs," in *Proceedings of the 18th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 46–57, IEEE, 1977.

[52] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps*. Portland, OR, USA: IT Revolution Press, 2018.

[53] DevOps Research and Assessment (DORA), "Accelerate: State of DevOps 2019." Google Cloud, 2019. `https://dora.dev/research/2019/`. Accessed August 2026.